

Random Search for Hyper-Parameter Optimization

James Bergstra

Yoshua Bengio

Département d'Informatique et de recherche opérationnelle

Université de Montréal

Montréal, QC, H3C 3J7, Canada

JAMES.BERGSTRA@UMONTREAL.CA

YOSHUA.BENGIO@UMONTREAL.CA

Editor: Leon Bottou

Abstract

Grid search and manual search are the most widely used strategies for hyper-parameter optimization. This paper shows empirically and theoretically that randomly chosen trials are more efficient for hyper-parameter optimization than trials on a grid. Empirical evidence comes from a comparison with a large previous study that used grid search and manual search to configure neural networks and deep belief networks. Compared with neural networks configured by a pure grid search, we find that random search over the same domain is able to find models that are as good or better within a small fraction of the computation time. Granting random search the same computational budget, random search finds better models by effectively searching a larger, less promising configuration space. Compared with deep belief networks configured by a thoughtful combination of manual search and grid search, purely random search over the same 32-dimensional configuration space found statistically equal performance on four of seven data sets, and superior performance on one of seven. A Gaussian process analysis of the function from hyper-parameters to validation set performance reveals that for most data sets only a few of the hyper-parameters really matter, but that different hyper-parameters are important on different data sets. This phenomenon makes grid search a poor choice for configuring algorithms for new data sets. Our analysis casts some light on why recent “High Throughput” methods achieve surprising success—they appear to search through a large number of hyper-parameters because most hyper-parameters do not matter much. We anticipate that growing interest in large hierarchical models will place an increasing burden on techniques for hyper-parameter optimization; this work shows that random search is a natural baseline against which to judge progress in the development of adaptive (sequential) hyper-parameter optimization algorithms.

Keywords: global optimization, model selection, neural networks, deep learning, response surface modeling

1. Introduction

The ultimate objective of a typical learning algorithm $\mathcal{A}$ is to find a function f that minimizes some expected loss $\mathcal{L}(x; f)$ over i.i.d. samples x from a natural (grand truth) distribution $\mathcal{G}_x$. A *learning algorithm* $\mathcal{A}$ is a functional that maps a data set $\mathcal{X}^{(\text{train})}$ (a finite set of samples from $\mathcal{G}_x$) to a function f . Very often a learning algorithm produces f through the optimization of a training criterion with respect to a set of *parameters* θ . However, the learning algorithm itself often has bells and whistles called *hyper-parameters* λ , and the actual learning algorithm is the one obtained after choosing λ , which can be denoted $\mathcal{A}_\lambda$, and $f = \mathcal{A}_\lambda(\mathcal{X}^{(\text{train})})$ for a training set $\mathcal{X}^{(\text{train})}$. For example, with a

Gaussian kernel SVM, one has to select a regularization penalty C for the training criterion (which controls the margin) and the bandwidth σ of the Gaussian kernel, that is, $\lambda = (C, \sigma)$.

What we really need in practice is a way to choose λ so as to minimize generalization error $\mathbb{E}_{x \sim \mathcal{G}_x}[\mathcal{L}(x; \mathcal{A}_\lambda(\mathcal{X}^{(\text{train})}))]$. Note that the computation performed by $\mathcal{A}$ itself often involves an inner optimization problem, which is usually iterative and approximate. The problem of identifying a good value for hyper-parameters λ is called the problem of *hyper-parameter optimization*. This paper takes a look at algorithms for this difficult outer-loop optimization problem, which is of great practical importance in empirical machine learning work:

$$\lambda^{(*)} = \underset{\lambda \in \Lambda}{\operatorname{argmin}} \mathbb{E}_{x \sim \mathcal{G}_x}[\mathcal{L}(x; \mathcal{A}_\lambda(\mathcal{X}^{(\text{train})}))]. \quad (1)$$

In general, we do not have efficient algorithms for performing the optimization implied by Equation 1. Furthermore, we cannot even evaluate the expectation over the unknown natural distribution $\mathcal{G}_x$, the value we wish to optimize. Nevertheless, we must carry out this optimization as best we can. With regards to the expectation over $\mathcal{G}_x$, we will employ the widely used technique of *cross-validation* to estimate it. Cross-validation is the technique of replacing the expectation with a mean over a *validation set* $\mathcal{X}^{(\text{valid})}$ whose elements are drawn i.i.d $x \sim \mathcal{G}_x$. Cross-validation is unbiased as long as $\mathcal{X}^{(\text{valid})}$ is independent of any data used by $\mathcal{A}_\lambda$ (see Bishop, 1995, pp. 32-33). We see in Equations 2-4 the hyper-parameter optimization problem as it is addressed in practice:

$$\lambda^{(*)} \approx \underset{\lambda \in \Lambda}{\operatorname{argmin}} \operatorname{mean}_{x \in \mathcal{X}^{(\text{valid})}} \mathcal{L}(x; \mathcal{A}_\lambda(\mathcal{X}^{(\text{train})})). \quad (2)$$

$$\equiv \underset{\lambda \in \Lambda}{\operatorname{argmin}} \Psi(\lambda) \quad (3)$$

$$\approx \underset{\lambda \in \{\lambda^{(1)} \dots \lambda^{(S)}\}}{\operatorname{argmin}} \Psi(\lambda) \equiv \hat{\lambda} \quad (4)$$

Equation 3 expresses the hyper-parameter optimization problem in terms of a *hyper-parameter response function*, Ψ . Hyper-parameter optimization is the minimization of $\Psi(\lambda)$ over $\lambda \in \Lambda$. This function is sometimes called the *response surface* in the experiment design literature. Different data sets, tasks, and learning algorithm families give rise to different sets Λ and functions Ψ . Knowing in general very little about the response surface Ψ or the search space Λ , the dominant strategy for finding a good λ is to choose some number (S) of *trial* points $\{\lambda^{(1)} \dots \lambda^{(S)}\}$, to evaluate $\Psi(\lambda)$ for each one, and return the $\lambda^{(i)}$ that worked the best as $\hat{\lambda}$. This strategy is made explicit by Equation 4.

The critical step in hyper-parameter optimization is to choose the set of trials $\{\lambda^{(1)} \dots \lambda^{(S)}\}$. The most widely used strategy is a combination of grid search and manual search (e.g., LeCun et al., 1998b; Larochelle et al., 2007; Hinton, 2010), as well as machine learning software packages such as libsvm (Chang and Lin, 2001) and scikits.learn.¹ If Λ is a set indexed by K configuration variables (e.g., for neural networks it would be the learning rate, the number of hidden units, the strength of weight regularization, etc.), then grid search requires that we choose a set of values for each variable ($L^{(1)} \dots L^{(K)}$). In grid search the set of trials is formed by assembling every possible combination of values, so the number of trials in a grid search is $S = \prod_{k=1}^K |L^{(k)}|$ elements. This product over K sets makes grid search suffer from the *curse of dimensionality* because the number of joint values grows exponentially with the number of hyper-parameters (Bellman, 1961). Manual

1. scikits.learn: Machine Learning in Python can be found at <http://scikit-learn.sourceforge.net>.

search is used to identify regions in Λ that are promising and to develop the intuition necessary to choose the sets $L^{(k)}$. A major drawback of manual search is the difficulty in *reproducing results*. This is important both for the progress of scientific research in machine learning as well as for ease of application of learning algorithms by non-expert users. On the other hand, grid search alone does very poorly in practice (as discussed here). We propose random search as a substitute and baseline that is both reasonably efficient (roughly equivalent to or better than combining manual search and grid search, in our experiments) and keeping the advantages of implementation simplicity and reproducibility of pure grid search. Random search is actually more practical than grid search because it can be applied even when using a cluster of computers that can fail, and allows the experimenter to change the “resolution” on the fly: adding new trials to the set or ignoring failed trials are both feasible because the trials are i.i.d., which is not the case for a grid search. Of course, random search can probably be improved by automating what manual search does, i.e., a sequential optimization, but this is left to future work.

There are several reasons why manual search and grid search prevail as the state of the art despite decades of research into global optimization (e.g., Nelder and Mead, 1965; Kirkpatrick et al., 1983; Powell, 1994; Weise, 2009) and the publishing of several hyper-parameter optimization algorithms (e.g., Nareyek, 2003; Czogiel et al., 2005; Hutter, 2009):

- Manual optimization gives researchers some degree of insight into Ψ ;
- There is no technical overhead or barrier to manual optimization;
- Grid search is simple to implement and parallelization is trivial;
- Grid search (with access to a compute cluster) typically finds a better $\hat{\lambda}$ than purely manual sequential optimization (in the same amount of time);
- Grid search is reliable in low dimensional spaces (e.g., 1-d, 2-d).

We will come back to the use of global optimization algorithms for hyper-parameter selection in our discussion of future work (Section 6). In this paper, we focus on random search, that is, independent draws from a uniform density from the same configuration space as would be spanned by a regular grid, as an alternative strategy for producing a trial set $\{\lambda^{(1)} \dots \lambda^{(S)}\}$. We show that random search has all the practical advantages of grid search (conceptual simplicity, ease of implementation, trivial parallelism) and trades a small reduction in efficiency in low-dimensional spaces for a large improvement in efficiency in high-dimensional search spaces.

In this work we show that random search is more efficient than grid search in high-dimensional spaces because functions Ψ of interest have a *low effective dimensionality*; essentially, Ψ of interest are more sensitive to changes in some dimensions than others (Caflisch et al., 1997). In particular, if a function f of two variables could be approximated by another function of one variable ($f(x_1, x_2) \approx g(x_1)$), we could say that f has a *low effective dimension*. Figure 1 illustrates how point grids and uniformly random point sets differ in how they cope with low effective dimensionality, as in the above example with f . A grid of points gives even coverage in the original 2-d space, but projections onto either the x_1 or x_2 subspace produces an inefficient coverage of the subspace. In contrast, random points are slightly less evenly distributed in the original space, but far more evenly distributed in the subspaces.

If the researcher could know ahead of time which subspaces would be important, then he or she could design an appropriate grid. However, we show the failings of this strategy in Section 2. For a

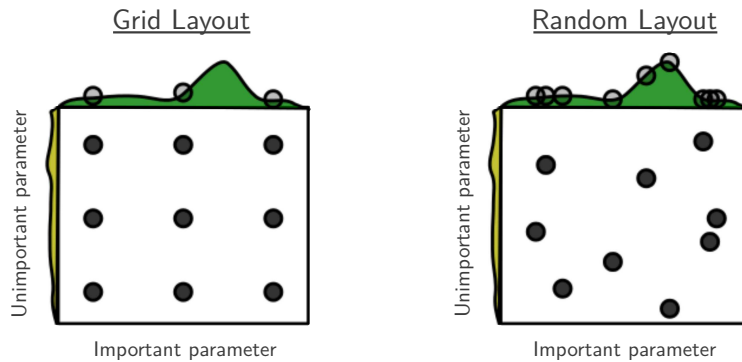


Figure 1: Grid and random search of nine trials for optimizing a function $f(x,y) = g(x) + h(y) \approx g(x)$ with low effective dimensionality. Above each square $g(x)$ is shown in green, and left of each square $h(y)$ is shown in yellow. With grid search, nine trials only test $g(x)$ in three distinct places. With random search, all nine trials explore distinct values of g . This failure of grid search is the rule rather than the exception in high dimensional hyper-parameter optimization.

given learning algorithm, looking at several relatively similar data sets (from different distributions) reveals that on different data sets, different subspaces are important, and to different degrees. A grid with sufficient granularity to optimizing hyper-parameters for all data sets must consequently be inefficient for each individual data set because of the curse of dimensionality: the number of wasted grid search trials is exponential in the number of search dimensions that turn out to be irrelevant for a particular data set. In contrast, random search thrives on low effective dimensionality. Random search has the same efficiency in the relevant subspace as if it had been used to search only the relevant dimensions.

This paper is organized as follows. Section 2 looks at the efficiency of random search in practice vs. grid search as a method for optimizing neural network hyper-parameters. We take the grid search experiments of Larochelle et al. (2007) as a point of comparison, and repeat similar experiments using random search. Section 3 uses Gaussian process regression (GPR) to analyze the results of the neural network trials. The GPR lets us characterize what Ψ looks like for various data sets, and establish an empirical link between the low effective dimensionality of Ψ and the efficiency of random search. Section 4 compares random search and grid search with more sophisticated point sets developed for Quasi Monte-Carlo numerical integration, and argues that in the regime of interest for hyper-parameter selection grid search is inappropriate and more sophisticated methods bring little advantage over random search. Section 5 compares random search with the expert-guided manual sequential optimization employed in Larochelle et al. (2007) to optimize Deep Belief Networks. Section 6 comments on the role of global optimization algorithms in future work. We conclude in Section 7 that random search is generally superior to grid search for optimizing hyper-parameters.

2. Random vs. Grid for Optimizing Neural Networks

In this section we take a second look at several of the experiments of Larochelle et al. (2007) using random search, to compare with the grid searches done in that work. We begin with a look at hyper-parameter optimization in neural networks, and then move on to hyper-parameter optimization in Deep Belief Networks (DBNs). To characterize the efficiency of random search, we present two techniques in preliminary sections: Section 2.1 explains how we estimate the generalization performance of the *best* model from a set of candidates, taking into account our uncertainty in which model is actually best; Section 2.2 explains the random experiment efficiency curve that we use to characterize the performance of random search experiments. With these preliminaries out of the way, Section 2.3 describes the data sets from Larochelle et al. (2007) that we use in our work. Section 2.4 presents our results optimizing neural networks, and Section 5 presents our results optimizing DBNs.

2.1 Estimating Generalization

Because of finite data sets, test error is not monotone in validation error, and depending on the set of particular hyper-parameter values λ evaluated, the test error of the best-validation error configuration may vary. When reporting performance of learning algorithms, it can be useful to take into account the uncertainty due to the choice of hyper-parameters values. This section describes our procedure for estimating test set accuracy, which takes into account any uncertainty in the choice of which trial is actually the best-performing one. To explain this procedure, we must distinguish between estimates of performance $\Psi^{(\text{valid})} = \Psi$ and $\Psi^{(\text{test})}$ based on the validation and test sets respectively:

$$\begin{aligned}\Psi^{(\text{valid})}(\lambda) &= \text{mean}_{x \in \mathcal{X}^{(\text{valid})}} \mathcal{L}(x; \mathcal{A}_\lambda(\mathcal{X}^{(\text{train})})), \\ \Psi^{(\text{test})}(\lambda) &= \text{mean}_{x \in \mathcal{X}^{(\text{test})}} \mathcal{L}(x; \mathcal{A}_\lambda(\mathcal{X}^{(\text{train})})).\end{aligned}$$

Likewise, we must define the estimated variance $\mathbb{V}$ about these means on the validation and test sets, for example, for the zero-one loss (Bernoulli variance):

$$\begin{aligned}\mathbb{V}^{(\text{valid})}(\lambda) &= \frac{\Psi^{(\text{valid})}(\lambda) (1 - \Psi^{(\text{valid})}(\lambda))}{|\mathcal{X}^{(\text{valid})}| - 1}, \text{ and} \\ \mathbb{V}^{(\text{test})}(\lambda) &= \frac{\Psi^{(\text{test})}(\lambda) (1 - \Psi^{(\text{test})}(\lambda))}{|\mathcal{X}^{(\text{test})}| - 1}.\end{aligned}$$

With other loss functions the estimator of variance will generally be different.

The standard practice for evaluating a model found by cross-validation is to report $\Psi^{(\text{test})}(\lambda^{(s)})$ for the $\lambda^{(s)}$ that minimizes $\Psi^{(\text{valid})}(\lambda^{(s)})$. However, when different trials have nearly optimal validation means, then it is not clear which test score to report, and a slightly different choice of λ could have yielded a different test error. To resolve the difficulty of choosing a winner, we report a weighted average of all the test set scores, in which each one is weighted by the probability that its particular $\lambda^{(s)}$ is in fact the best. In this view, the uncertainty arising from $\mathcal{X}^{(\text{valid})}$ being a finite sample of $\mathcal{G}_x$ makes the test-set score of the best model among $\lambda^{(1)}, \dots, \lambda^{(S)}$ a random variable, z . This score z is modeled by a Gaussian mixture model whose S components have means $\mu_s = \Psi^{(\text{test})}(\lambda^{(s)})$,

variances $\sigma_s^2 = \mathbb{V}^{(\text{test})}(\lambda^{(s)})$, and weights w_s defined by

$$w_s = P\left(Z^{(s)} < Z^{(s')}, \forall s' \neq s\right), \text{ where}$$

$$Z^{(i)} \sim \mathcal{N}\left(\Psi^{(\text{valid})}(\lambda^{(i)}), \mathbb{V}^{(\text{valid})}(\lambda^{(i)})\right).$$

To summarize, the performance z of the best model in an experiment of S trials has mean μ_z and standard error σ_z^2 ,

$$\mu_z = \sum_{s=1}^S w_s \mu_s, \text{ and} \quad (5)$$

$$\sigma_z^2 = \sum_{s=1}^S w_s (\mu_s^2 + \sigma_s^2) - \mu_z^2. \quad (6)$$

It is simple and practical to estimate weights w_s by simulation. The procedure for doing so is to repeatedly draw hypothetical validation scores $Z^{(s)}$ from Normal distributions whose means are the $\Psi^{(\text{valid})}(\lambda^{(s)})$ and whose variances are the squared standard errors $\mathbb{V}^{(\text{valid})}(\lambda^{(s)})$, and to count how often each trial generates a winning score. Since the test scores of the best validation scores are typically relatively close, w_s need not be estimated very precisely and a few tens of hypothetical draws suffice.

In expectation, this technique for estimating generalization gives a higher estimate than the traditional technique of reporting the test set error of the best model in validation. The difference is related to the variance $\Psi^{(\text{valid})}$ and the density of validation set scores $\Psi(\lambda^{(i)})$ near the best value. To the extent that $\Psi^{(\text{valid})}$ casts doubt on which model was best, this technique averages the performance of the best model together with the performance of models which were not the best. The next section (Random Experiment Efficiency Curve) illustrates this phenomenon and discusses it in more detail.

2.2 Random Experiment Efficiency Curve

Figure 2 illustrates the results of a random experiment: an experiment of 256 trials training neural networks to classify the rectangles data set. Since the trials of a random experiment are independently identically distributed (i.i.d.), a random search experiment involving S i.i.d. trials can also be interpreted as N independent experiments of s trials, as long as $sN \leq S$. This interpretation allows us to estimate statistics such as the minimum, maximum, median, and quantiles of any random experiment of size s , where s is a divisor of S .

There are two general trends in random experiment efficiency curves, such as the one in Figure 2: a sharp upward slope of the lower extremes as experiments grow, and a gentle downward slope of the upper extremes. The sharp upward slope occurs because when we take the maximum over larger subsets of the S trials, trials with poor performance are rarely the best within their subset. It is natural that larger experiments find trials with better scores. The shape of this curve indicates the frequency of good models under random search, and quantifies the relative volumes (in search space) of the various levels of performance.

The gentle downward slope occurs because as we take the maximum over larger subsets of trials (in Equation 6), we are less sure about which trial is actually the best. Large experiments average together good validation trials with unusually high test scores with other good validation trials with unusually low test scores to arrive at a more accurate estimate of generalization. For example,

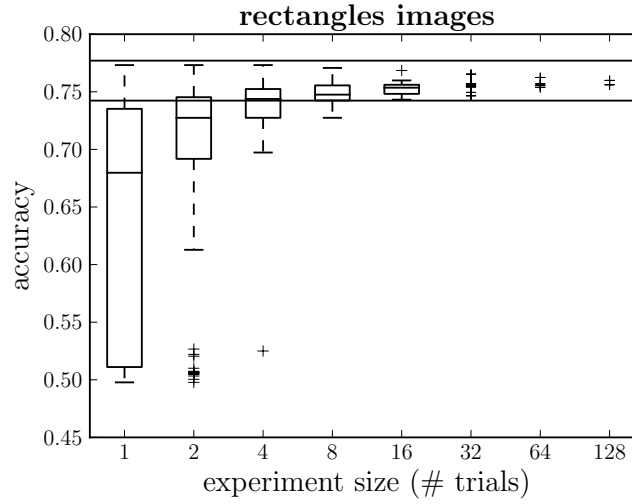


Figure 2: A random experiment efficiency curve. The trials of a random experiment are i.i.d, so an experiment of many trials (here, 256 trials optimizing a neural network to classify the **rectangles basic** data set, Section 2.3) can be interpreted as several independent smaller experiments. For example, at horizontal axis position 8, we consider our 256 trials to be 32 experiments of 8 trials each. The vertical axis shows the test accuracy of the best trial(s) from experiments of a given size, as determined by Equation 5. When there are sufficiently many experiments of a given size (i.e., 10), the distribution of performance is illustrated by a box plot whose boxed section spans the lower and upper quartiles and includes a line at the median. The whiskers above and below each boxed section show the position of the most extreme data point within 1.5 times the inter-quartile range of the nearest quartile. Data points beyond the whiskers are plotted with '+' symbols. When there are not enough experiments to support a box plot, as occurs here for experiments of 32 trials or more, the best generalization score of each experiment is shown by a scatter plot. The two thin black lines across the top of the figure mark the upper and lower boundaries of a 95% confidence interval on the generalization of the best trial overall (Equation 6).

consider what Figure 2 would look like if the experiment had included *lucky trial* whose validation score were around 77% as usual, but whose test score were 80%. In the bar plot for trials of size 1, we would see the top performer scoring 80%. In larger experiments, we would average that 80% performance together with other test set performances because 77% is not clearly the best validation score; this averaging would make the upper envelope of the efficiency curve slope downward from 80% to a point very close to the current test set estimate of 76%.

Figure 2 characterizes the range of performance that is to be expected from experiments of various sizes, which is valuable information to anyone trying to reproduce these results. For example, if we try to repeat the experiment and our first four random trials fail to find a score better than 70%, then the problem is likely not in hyper-parameter selection.

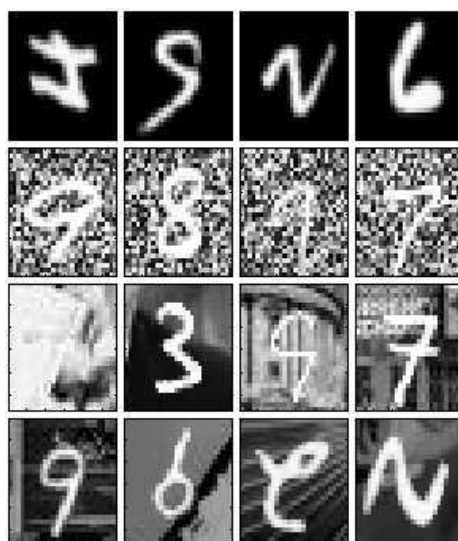


Figure 3: From top to bottom, samples from the **mnist rotated**, **mnist background random**, **mnist background images**, **mnist rotated background images** data sets. In all data sets the task is to identify the digit (0 - 9) and ignore the various distracting factors of variation.

2.3 Data Sets

Following the work of Larochelle et al. (2007) and Vincent et al. (2008), we use a variety of classification data sets that include many factors of variation.²

The **mnist basic** data set is a subset of the well-known MNIST handwritten digit data set (LeCun et al., 1998a). This data set has 28x28 pixel grey-scale images of digits, each belonging to one of ten classes. We chose a different train/test/validation splitting in order to have faster experiments and see learning performance differences more clearly. We shuffled the original splits randomly, and used 10 000 training examples, 2000 validation examples, and 50 000 testing examples. These images are presented as white (1.0-valued) foreground digits against a black (0.0-valued) background.

The **mnist background images** data set is a variation on **mnist basic** in which the white foreground digit has been composited on top of a 28x28 natural image patch. Technically this was done by taking the maximum of the original MNIST image and the patch. Natural image patches with very low pixel variance were rejected. As with **mnist basic** there are 10 classes, 10 000 training examples, 2000 validation examples, and 50 000 test examples.

The **mnist background random** data set is a similar variation on **mnist basic** in which the white foreground digit has been composited on top of random uniform (0,1) pixel values. As with **mnist basic** there are 10 classes, 10 000 training examples, 2000 validation examples, and 50 000 test examples.

The **mnist rotated** data set is a variation on **mnist basic** in which the images have been rotated by an amount chosen randomly between 0 and 2π radians. This data set included 10000 training examples, 2000 validation examples, 50 000 test examples.

2. Data sets can be found at <http://www.iro.umontreal.ca/~lisa/twiki/bin/view.cgi/Public/DeepVsShallowComparisonICML2007>.

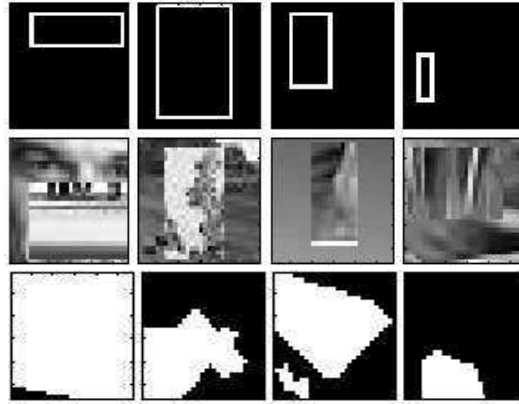


Figure 4: *Top*: Samples from the **rectangles** data set. *Middle*: Samples from the **rectangles images** data set. *Bottom*: Samples from the **convex** data set. In rectangles data sets, the image is formed by overlaying a small rectangle on a background. The task is to label the small rectangle as being either tall or wide. In **convex**, the task is to identify whether the set of white pixels is convex (images 1 and 4) or not convex (images 2 and 3).

The **mnist rotated background images** data set is a variation on **mnist rotated** in which the images have been rotated by an amount chosen randomly between 0 and 2π radians, and then subsequently composited onto natural image patch backgrounds. This data set included 10000 training examples, 2000 validation examples, 50 000 test examples.

The **rectangles** data set (Figure 4, top) is a simple synthetic data set of outlines of rectangles. The images are 28x28, the outlines are white (1-valued) and the backgrounds are black (0-valued). The height and width of the rectangles were sampled uniformly, but when their difference was smaller than 3 pixels the samples were rejected. The top left corner of the rectangles was also sampled uniformly, with the constraint that the whole rectangle fits in the image. Each image is labelled as one of two classes: tall or wide. This task was easier than the MNIST digit classification, so we only used 1000 training examples, and 200 validation examples, but we still used 50 000 testing examples.

The **rectangles images** data set (Figure 4, middle) is a variation on **rectangles** in which the foreground rectangles were filled with one natural image patch, and composited on top of a different background natural image patch. The process for sampling rectangle shapes was similar to the one used for **rectangles**, except a) the area covered by the rectangles was constrained to be between 25% and 75% of the total image, b) the length and width of the rectangles were forced to be of at least 10 pixels, and c) their difference was forced to be of at least 5 pixels. This task was harder than **rectangles**, so we used 10000 training examples, 2000 validation examples, and 50 000 testing examples.

The **convex** data set (Figure 4, bottom) is a binary image classification task. Each 28x28 image consists entirely of 1-valued and 0-valued pixels. If the 1-valued pixels form a convex region in image space, then the image is labelled as being convex, otherwise it is labelled as non-convex. The convex sets consist of a single convex region with pixels of value 1.0. Candidate convex images were constructed by taking the intersection of a number of half-planes whose location and orienta-

tion were chosen uniformly at random. The number of intersecting half-planes was also sampled randomly according to a geometric distribution with parameter 0.195. A candidate convex image was rejected if there were less than 19 pixels in the convex region. Candidate non-convex images were constructed by taking the union of a random number of convex sets generated as above, but with the number of half-planes sampled from a geometric distribution with parameter 0.07 and with a minimum number of 10 pixels. The number of convex sets was sampled uniformly from 2 to 4. The candidate non-convex images were then tested by checking a convexity condition for every pair of pixels in the non-convex set. Those sets that failed the convexity test were added to the data set. The parameters for generating the convex and non-convex sets were balanced to ensure that the conditional overall pixel mean is the same for both classes.

2.4 Case Study: Neural Networks

In Larochelle et al. (2007), the hyper-parameters of the neural network were optimized by search over a grid of trials. We describe the hyper-parameter configuration space of our neural network learning algorithm in terms of the distribution that we will use to randomly sample from that configuration space. The first hyper-parameter in our configuration is the type of data preprocessing: with equal probability, one of (a) none, (b) normalize (center each feature dimension and divide by its standard deviation), or (c) PCA (after removing dimension-wise means, examples are projected onto principle components of the data whose norms have been divided by their eigenvalues). Part of PCA preprocessing is choosing how many components to keep. We choose a fraction of variance to keep with a uniform distribution between 0.5 and 1.0. There have been several suggestions for how the random weights of a neural network should be initialized (we will look at unsupervised learning *pretraining* algorithms later in Section 5). We experimented with two distributions and two scaling heuristics. The possible distributions were (a) uniform on $(-1, 1)$, and (b) unit normal. The two scaling heuristics were (a) a hyper-parameter multiplier between 0.1 and 10.0 divided by the square root of the number of inputs (LeCun et al., 1998b), and (b) the square root of 6 divided by the square root of the number of inputs plus hidden units (Bengio and Glorot, 2010). The weights themselves were chosen using one of three random seeds to the Mersenne Twister pseudo-random number generator. In the case of the first heuristic, we chose a multiplier uniformly from the range $(0.2, 2.0)$. The number of hidden units was drawn geometrically³ from 18 to 1024. We selected either a sigmoidal or tanh nonlinearity with equal probability. The output weights from hidden units to prediction units were initialized to zero. The cost function was the mean error over minibatches of either 20 or 100 (with equal probability) examples at a time: in expectation these give the same gradient directions, but with more or less variance. The optimization algorithm was stochastic gradient descent with [initial] learning rate ϵ_0 drawn geometrically from 0.001 to 10.0. We offered the possibility of an annealed learning rate via a time point t_0 drawn geometrically from 300 to 30000. The effective learning rate ϵ_t after t minibatch iterations was

$$\epsilon_t = \frac{t_0 \epsilon_0}{\max(t, t_0)}. \quad (7)$$

We permitted a minimum of 100 and a maximum of 1000 iterations over the training data, stopping if ever, at iteration t , the best validation performance was observed before iteration $t/2$. With 50%

3. We will use the phrase *drawn geometrically* from A to B for $0 < A < B$ to mean drawing uniformly in the log domain between $\log(A)$ and $\log(B)$, exponentiating to get a number between A and B , and then rounding to the nearest integer. The phrase *drawn exponentially* means the same thing but without rounding.

probability, an ℓ_2 regularization penalty was applied, whose strength was drawn exponentially from 3.1×10^{-7} to 3.1×10^{-5} . This sampling process covers roughly the same domain with the same density as the grid used in Larochelle et al. (2007), except for the optional preprocessing steps. The grid optimization of Larochelle et al. (2007) did not consider normalizing or keeping only leading PCA dimensions of the inputs; we compare to random sampling with and without these restrictions.⁴

We formed experiments for each data set by drawing $S = 256$ trials from this distribution. The results of these experiments are illustrated in Figures 5 and 6. Random sampling of trials is surprisingly effective in these settings. Figure 5 shows that even among the fraction of jobs (71/256) that used no preprocessing, the random search with 8 trials is better than the grid search employed in Larochelle et al. (2007).

Typically, the extent of a grid search is determined by a computational budget. Figure 6 shows what is possible if we use random search in a larger space that requires more trials to explore. The larger search space includes the possibility of normalizing the input or applying PCA preprocessing. In the larger space, 32 trials were necessary to consistently outperform grid search rather than 8, indicating that there are many harmful ways to preprocess the data. However, when we allowed larger experiments of 64 trials or more, random search found superior results to those found more quickly within the more restricted search. This tradeoff between exploration and exploitation is central to the design of an effective random search.

The efficiency curves in Figures 5 and 6 reveal that different data sets give rise to functions Ψ with different shapes. The **mnist basic** results converge very rapidly toward what appears to be a global maximum. The fact that experiments of just 4 or 8 trials often have the same maximum as much larger experiments indicates that the region of Λ that gives rise to the best performance is approximately a quarter or an eighth respectively of the entire configuration space. Assuming that the random search has not missed a tiny region of significantly better performance, we can say that random search has solved this problem in 4 or 8 guesses. It is hard to imagine any optimization algorithm doing much better on a non-trivial 7-dimensional function. In contrast the **mnist rotated background images** and **convex** curves show that even with 16 or 32 random trials, there is considerable variation in the generalization of the reportedly best model. This indicates that the Ψ function in these cases is more peaked, with small regions of good performance.

3. The Low Effective Dimension of Ψ

Section 2 showed that random sampling is more efficient than grid sampling for optimizing functions Ψ corresponding to several neural network families and classification tasks. In this section we show that indeed Ψ has a low effective dimension, which explains why randomly sampled trials found better values. One simple way to characterize the shape of a high-dimensional function is to look at how much it varies in each dimension. Gaussian process regression gives us the statistical machinery to look at Ψ and measure its effective dimensionality (Neal, 1998; Rasmussen and Williams, 2006).

We estimated the sensitivity of Ψ to each hyper-parameter by fitting a Gaussian process (GP) with squared exponential kernels to predict $\Psi(\lambda)$ from λ . The squared exponential kernel (or Gaussian kernel) measures similarity between two real-valued hyper-parameter values a and b by $\exp(-(\frac{a-b}{l})^2)$. The positive-valued l governs the sensitivity of the GP to change in this hyper-

4. Source code for the simulations is available at <https://github.com/jaberg/hyperopt>.

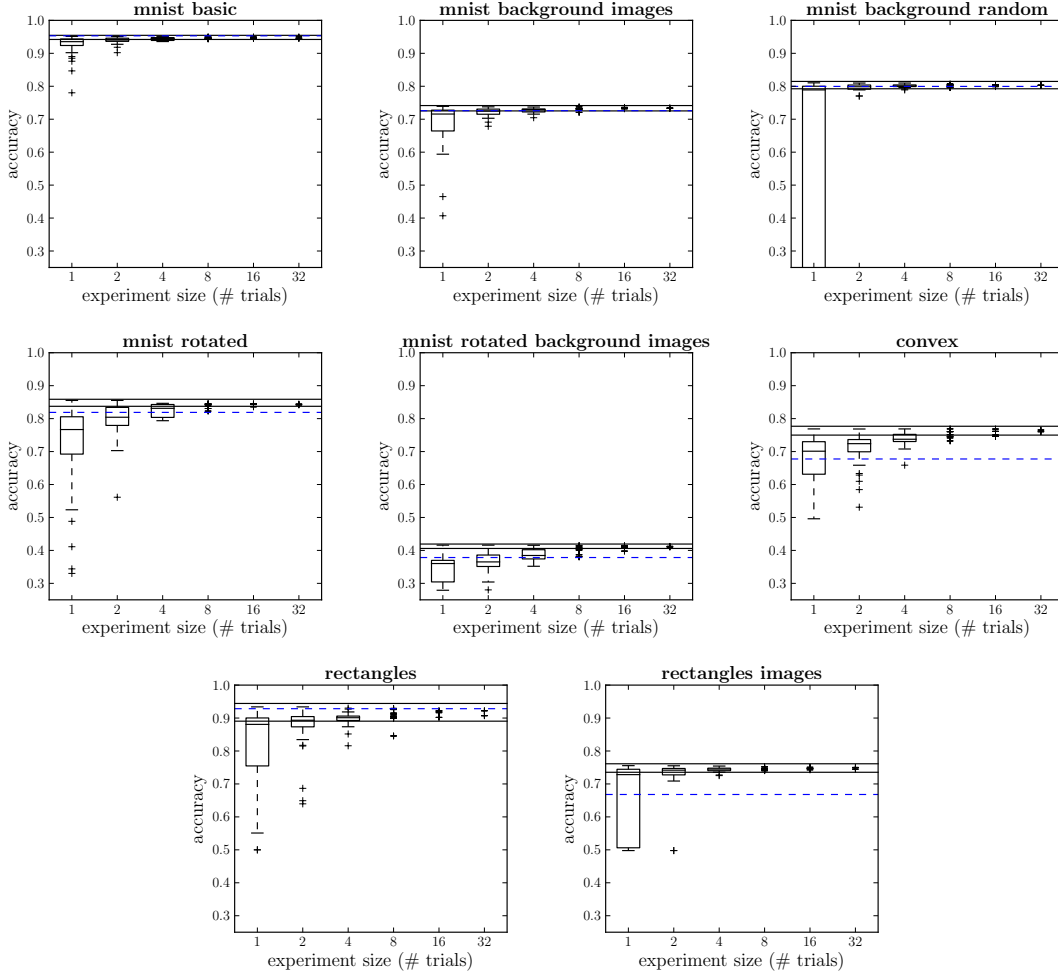


Figure 5: Neural network performance without preprocessing. Random experiment efficiency curves of a single-layer neural network for eight of the data sets used in Larochelle et al. (2007), looking only at trials with no preprocessing (7 hyper-parameters to optimize). The vertical axis is test-set accuracy of the best model by cross-validation, the horizontal axis is the experiment size (the number of models compared in cross-validation). The dashed blue line represents grid search accuracy for neural network models based on a selection by grids averaging 100 trials (Larochelle et al., 2007). Random searches of 8 trials match or outperform grid searches of (on average) 100 trials.

parameter. The kernels defined for each hyper-parameter were combined by multiplication (joint Gaussian kernel). We fit a GP to samples of Ψ by finding the *length scale* (l) for each hyper-parameter that maximized the marginal likelihood. To ensure relevance could be compared between hyper-parameters, we shifted and scaled each one to the unit interval. For hyper-parameters that were drawn geometrically or exponentially (e.g., learning rate, number of hidden units), kernel calculations were based on the logarithm of the effective value.

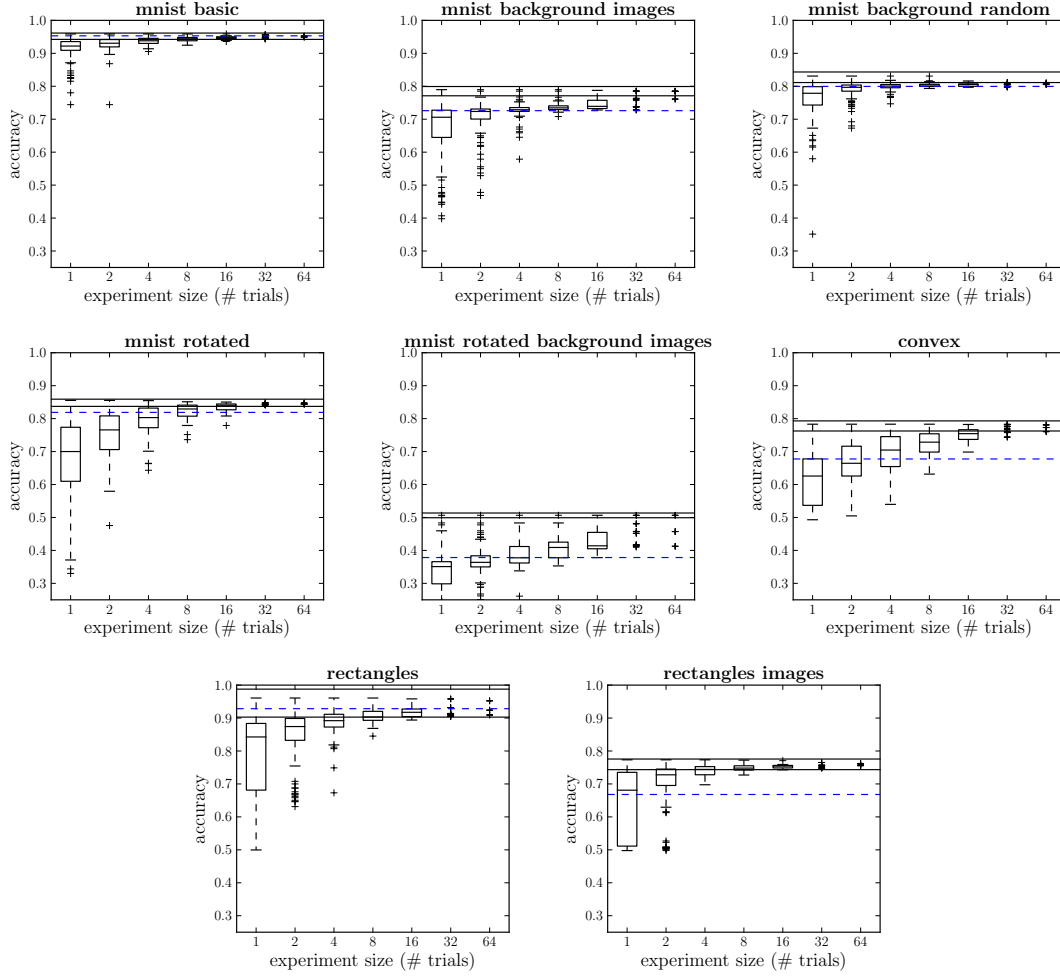


Figure 6: Neural network performance when standard preprocessing algorithms are considered (9 hyper-parameters). Dashed blue line represents grid search accuracy using (on average) 100 trials (Larochelle et al., 2007), in which no preprocessing was done. Often the extent of a search is determined by a computational budget, and with random search 64 trials are enough to find better models in a larger less promising space. Exploring just four PCA variance levels by grid search would have required 5 times as many (average 500) trials per data set.

Figure 7 shows the relevance of each component of Λ in modelling $\Psi(\lambda)$. Finding the length scales that maximize marginal likelihood is not a convex problem and many local minima exist. To get a sense of what length scales were supported by the data, we fit each set of samples from Ψ 50 times, resampling different subsets of 80% of the observations every time, and reinitializing the length scale estimates randomly between 0.1 and 2. Figure 7 reveals two important properties of Ψ for neural networks that suggest why grid search performs so poorly relative to random experiments:

1. a small fraction of hyper-parameters matter for any one data set, but

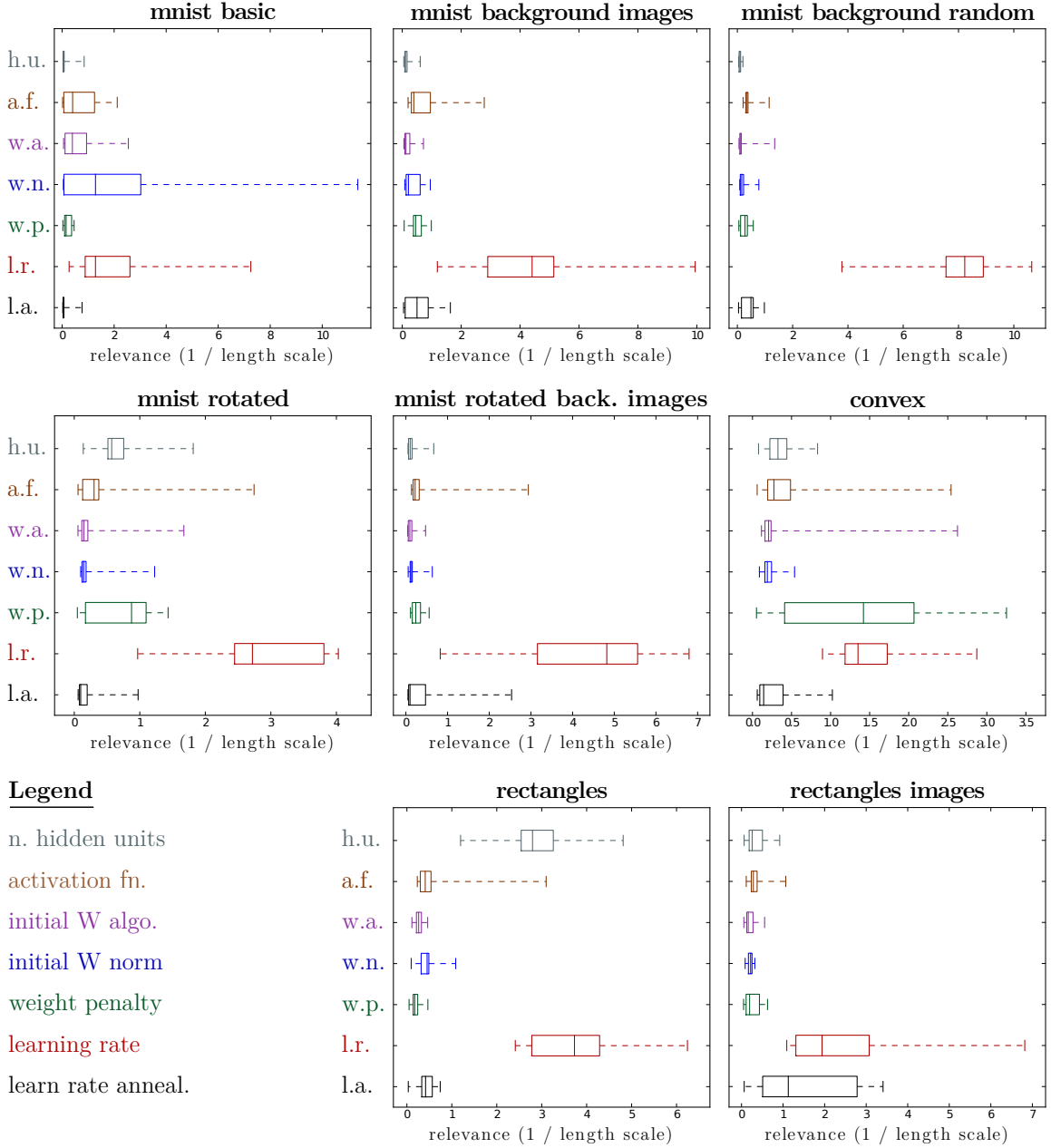


Figure 7: Automatic Relevance Determination (ARD) applied to hyper-parameters of neural network experiments (with raw preprocessing). For each data set, a small number of hyper-parameters dominate performance, but the relative importance of each hyper-parameter varies from each data set to the next. Section 2.4 describes the seven hyper-parameters in each panel. Boxplots are obtained by randomizing the subset of data used to fit the length scales, and randomizing the length scale initialization. (*Best viewed in color.*)

2. different hyper-parameters matter on different data sets.

Even in this simple 7-d problem, Ψ has a much lower effective dimension of between 1 and 4, depending on the data set. It would be impossible to cover just these few dimensions with a reliable grid however, because different data sets call for grids on different dimensions. The learning rate is always important, but sometimes the learning rate annealing rate was important (**rectangles images**), sometimes the ℓ_2 -penalty was important (**convex**, **mnist rotated**), sometimes the number of hidden units was important (**rectangles**), and so on. While random search optimized these Ψ functions with 8 to 16 trials, a grid with, say, four values in each of these axes would already require 256 trials, and yet provide no guarantee that Ψ for a new data set would be well optimized.

Figure 7 also allows us to establish a correlation between effective dimensionality and ease of optimization. The data sets for which the effective dimensionality was lowest (1 or 2) were **mnist basic**, **mnist background images**, **mnist background random**, and **rectangles images**. Looking back at the corresponding efficiency curves (Figure 5) we find that these are also the data sets whose curves plateau most sharply, indicating that these functions are the easiest to optimize. They are often optimized reasonably well by just 2 random trials. Looking to Figure 7 at the data sets with largest effective dimensionality (3 or 4), we identify **convex**, **mnist rotated**, **rectangles**. Looking at their efficiency curves in Figure 5 reveals that they consistently required at least 8 random trials. This correlation offers another piece of evidence that the effective dimensionality of Ψ is playing a strong role in determining the difficulty of hyper-parameter optimization.

4. Grid Search and Sets with Low Effective Dimensionality

It is an interesting mathematical challenge to choose a set of trials for sampling functions of unknown, but low effective dimensionality. We would like it to be true that no matter which dimensions turn out to be important, our trials sample the important dimensions evenly. Sets of points with this property are well studied in the literature of Quasi-Random methods for numerical integration, where they are known as *low-discrepancy sets* because they try to match (minimize discrepancy with) the uniform distribution. Although there are several formal definitions of low discrepancy, they all capture the intuition that the points should be roughly equidistant from one another, in order that there be no “clumps” or “holes” in the point set.

Several procedures for constructing low-discrepancy point sets in multiple dimensions also try to ensure as much as possible that subspace projections remain low-discrepancy sets in the subspace. For example, the Sobol (Antonov and Saleev, 1979), Halton (Halton, 1960), and Niederreiter (Bratley et al., 1992) sequences, as well as latin hypercube sampling (McKay et al., 1979) are all more or less deterministic schemes for getting point sets that are more representative of random uniform draws than actual random uniform draws. In Quasi Monte-Carlo integration, such point sets are shown to asymptotically minimize the variance of finite integrals faster than true random uniform samples, but in this section, we will look at these point sets in the setting of relatively small sample sizes, to see if they can be used for more efficient search than random draws.

Rather than repeat the very computationally expensive experiments conducted in Section 2, we used an artificial simulation to compare the efficiency of grids, random draws, and the four low-discrepancy point sets mentioned in the previous paragraph. The artificial search problem was to find a uniformly randomly placed multi-dimensional target interval, which occupies 1% of the volume of the unit hyper-cube. We looked at four variants of the search problem, in which the target was

1. a cube in a 3-dimensional space,
2. a hyper-rectangle in a 3-dimensional space,
3. a hyper-cube in a 5-dimensional space,
4. a hyper-rectangle in a 5-dimensional space.

The shape of the target rectangle in variants (2) and (4) was determined by sampling side lengths uniformly from the unit interval, and then scaling the rectangle to have a volume of 1%. This process gave the rectangles a shape that was often wide or tall - much longer along some axes than others. The position of the target was drawn uniformly among the positions totally inside the unit hyper-cube. In the case of tall or wide targets (2) and (4), the indicator function [of the target] had a lower effective dimension than the dimensionality of the overall space because the dimensions in which the target is elongated can be almost ignored.

The simulation experiment began with the generation of 100 random search problems. Then for each experiment design method (random, Sobol, latin hypercube, grid) we created experiments of 1, 2, 3, and so on up to 512 trials.⁵ The Sobol, Niederreiter, and Halton sequences yielded similar results, so we used the Sobol sequence to represent the performance of these low-discrepancy set construction methods. There are many possible grid experiments of any size in multiple dimensions (at least for non-prime experiment sizes). We did not test every possible grid, instead we tested every grid with a monotonic resolution. For example, for experiments of size 16 in 5 dimensions we tried the five grids with resolutions (1, 1, 1, 1, 16), (1, 1, 1, 2, 8), (1, 1, 2, 2, 4), (1, 1, 1, 4, 4), (1, 2, 2, 2, 2); for experiments of some prime size P in 3 dimensions we tried one grid with resolution (1, 1, P). Since the target intervals were generated in such a way that rectangles identical up to a permutation of side lengths have equal probability, grids with monotonic resolution are representative of all grids. The score of an experiment design method for each experiment size was the fraction of the 100 targets that it found.

To characterize the performance of random search, we used the analytic form of the expectation. The expected probability of finding the target is 1.0 minus the probability of missing the target with every single one of T trials in the experiment. If the volume of the target relative to the unit hypercube is ($v/V = 0.01$) and there are T trials, then this probability of finding the target is

$$1 - (1 - \frac{v}{V})^T = 1 - 0.99^T.$$

Figure 8 illustrates the efficiency of each kind of point set at finding the multidimensional intervals. There were some grids that were best at finding cubes and hyper-cubes in 3-d and 5-d, but most grids were the worst performers. No grid was competitive with the other methods at finding the rectangular-shaped intervals, which had low effective dimension (cases 2 and 4; Figure 8, right panels). Latin hypercubes, commonly used to initialize experiments in Bayesian optimization, were no more efficient than the expected performance of random search. Interestingly, the Sobol sequence was consistently best by a few percentage points. The low-discrepancy property that makes the Sobol useful in integration helps here, where it has the effect of minimizing the size of holes where the target might pass undetected. The advantage of the Sobol sequence is most pronounced in experiments of 100-300 trials, where there are sufficiently many trials for the structure in the Sobol

5. Samples from the Sobol sequence were provided by the GNU Scientific Library (M. Galassi et al., 2009).

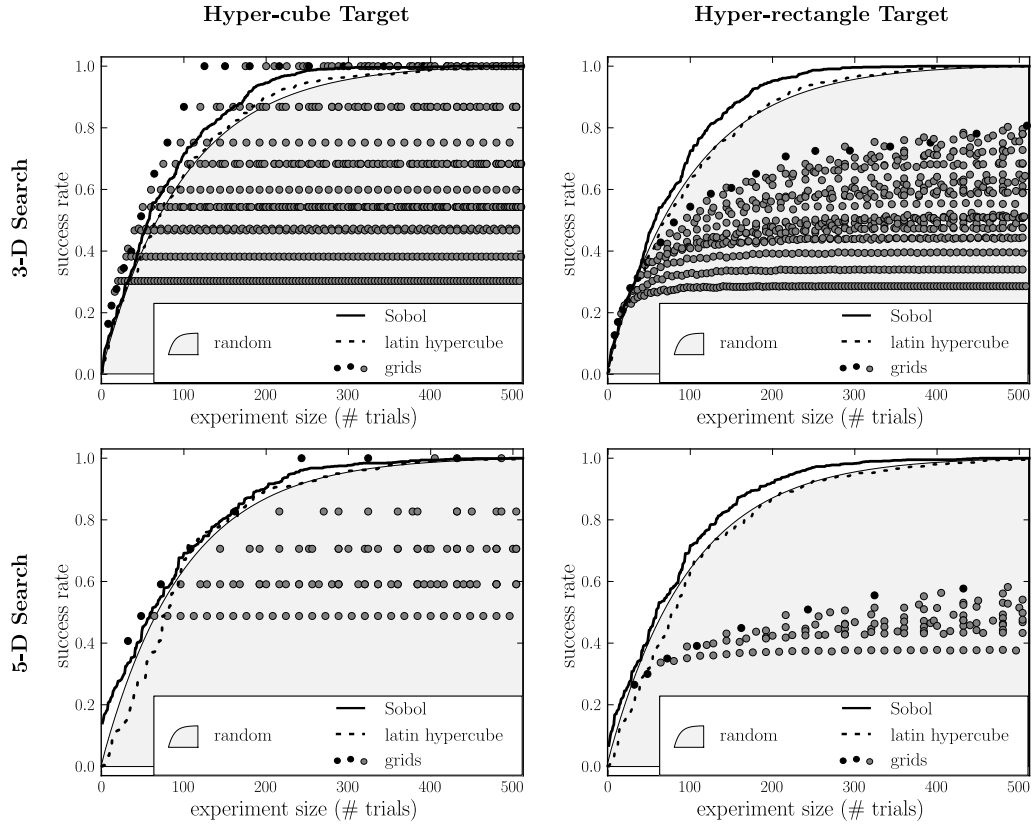


Figure 8: The efficiency in simulation of low-discrepancy sequences relative to grid and pseudo-random experiments. The simulation tested how reliably various experiment design methods locate a multidimensional interval occupying 1% of a unit hyper-cube. There is one grey dot in each sub-plot for every grid of every experiment size that has at least two ticks in each dimension. The black dots indicate near-perfect grids whose finest and coarsest dimensional resolutions differ by either 0 or 1. Hyper-parameter search is most typically like the bottom-right scenario. Grid search experiments are inefficient for finding axis-aligned elongated regions in high dimensions (i.e., bottom-right). Pseudo-random samples are as efficient as latin hypercube samples, and slightly less efficient than the Sobol sequence.

depart significantly from i.i.d points, but not sufficiently many trials for random search to succeed with high probability.

A thought experiment gives some intuition for why grid search fails in the case of rectangles. Long thin rectangles tend to intersect with several points if they intersect with any, reducing the effective sample size of the search. If the rectangles had been rotated away from the axes used to build the grid, then depending on the angle the efficiency of grid could approach the efficiency of random or low-discrepancy trials. More generally, if the target manifold were not systematically aligned with subsets of trial points, then grid search would be as efficient as the random and quasi-random searches.

5. Random Search vs. Sequential Manual Optimization

To see how random search compares with a careful combination of grid search and hand-tuning in the context of a model with many hyper-parameters, we performed experiments with the Deep Belief Network (DBN) model (Hinton et al., 2006). A DBN is a multi-layer graphical model with directed and undirected components. It is parameterized similarly to a multilayer neural network for classification, and it has been argued that *pretraining* a multilayer neural network by unsupervised learning as a DBN acts both to regularize the neural network toward better generalization, and to ease the optimization associated with *finetuning* the neural network for a classification task (Erhan et al., 2010).

A DBN classifier has many more hyper-parameters than a neural network. Firstly, there is the number of units and the parameters of random initialization for each layer. Secondly, there are hyper-parameters governing the unsupervised pretraining algorithm for each layer. Finally, there are hyper-parameters governing the global finetuning of the whole model for classification. For the details of how DBN models are trained (stacking restricted Boltzmann machines trained by contrastive divergence), the reader is referred to Larochelle et al. (2007), Hinton et al. (2006) or Bengio (2009). We evaluated random search by training 1-layer, 2-layer and 3-layer DBNs, sampling from the following distribution:

- We chose 1, 2, or 3 layers with equal probability.
- For each layer, we chose:
 - a number of hidden units (log-uniformly between 128 and 4000),
 - a weight initialization heuristic that followed from a distribution (uniform or normal), a multiplier (uniformly between 0.2 and 2), a decision to divide by the fan-out (true or false),
 - a number of iterations of contrastive divergence to perform for pretraining (log-uniformly from 1 to 10000),
 - whether to treat the real-valued examples used for unsupervised pretraining as Bernoulli means (from which to draw binary-valued training samples) or as samples themselves (even though they are not binary),
 - an initial learning rate for contrastive divergence (log-uniformly between 0.0001 and 1.0),
 - a time point at which to start annealing the contrastive divergence learning rate as in Equation 7 (log-uniformly from 10 to 10 000).
- There was also the choice of how to preprocess the data. Either we used the raw pixels or we removed some of the variance using a ZCA transform (in which examples are projected onto principle components, and then multiplied by the transpose of the principle components to place them back in the inputs space).
- If using ZCA preprocessing, we kept an amount of variance drawn uniformly from 0.5 to 1.0.
- We chose to seed our random number generator with one of 2, 3, or 4.
- We chose a learning rate for finetuning of the final classifier log-uniformly from 0.001 to 10.

- We chose an anneal start time for finetuning log-uniformly from 100 to 10000.
- We chose ℓ_2 regularization of the weight matrices at each layer during finetuning to be either 0 (with probability 0.5), or log-uniformly from 10^{-7} to 10^{-4} .

This hyper-parameter space includes 8 global hyper-parameters and 8 hyper-parameters for each layer, for a total of 32 hyper-parameters for 3-layer models.

A grid search is not practical for the 32-dimensional search problem of DBN model selection, because even just 2 possible values for each of 32 hyper-parameters would yield more trials than we could conduct ($2^{32} > 10^9$ trials and each can take hours). For many of the hyper-parameters, especially real valued ones, we would really like to try more than two values. The approach taken in Larochelle et al. (2007) was a combination of manual search, multi-resolution grid search and coordinate descent. The algorithm (including manual steps) is somewhat elaborate, but sensible, and we believe that it is representative of how model search is typically done in several research groups, if not the community at large. Larochelle et al. (2007) describe it as follows:

“The hyper-parameter search procedure we used alternates between fixing a neural network architecture and searching for good optimization hyper-parameters similarly to coordinate descent. More time would usually be spent on finding good optimization parameters, given some empirical evidence that we found indicating that the choice of the optimization hyper-parameters (mostly the learning rates) has much more influence on the obtained performance than the size of the network. We used the same procedure to find the hyper-parameters for DBN-1, which are the same as those of DBN-3 except the second hidden layer and third hidden layer sizes. We also allowed ourselves to test for much larger first-hidden layer sizes, in order to make the comparison between DBN-1 and DBN-3 fairer.

“We usually started by testing a relatively small architecture (between 500 and 700 units in the first and second hidden layer, and between 1000 and 2000 hidden units in the last layer). Given the results obtained on the validation set (compared to those of NNet for instance) after selecting appropriate optimization parameters, we would then consider growing the number of units in all layers simultaneously. The biggest networks we eventually tested had up to 3000, 4000 and 6000 hidden units in the first, second and third hidden layers respectively.

“As for the optimization hyper-parameters, we would proceed by first trying a few combinations of values for the stochastic gradient descent learning rate of the supervised and unsupervised phases (usually between 0.1 and 0.0001). We then refine the choice of tested values for these hyper-parameters. The first trials would simply give us a trend on the validation set error for these parameters (is a change in the hyper-parameter making things worse or better) and we would then consider that information in selecting appropriate additional trials. One could choose to use learning rate adaptation techniques (e.g., slowly decreasing the learning rate or using momentum) but we did not find these techniques to be crucial.

There was large variation in the number of trials used in Larochelle et al. (2007) to optimize the DBN-3. One data set (**mnist background images**) benefited from 102 trials, while another (**mnist background random**) only 13 because a good result was found more quickly. The average number

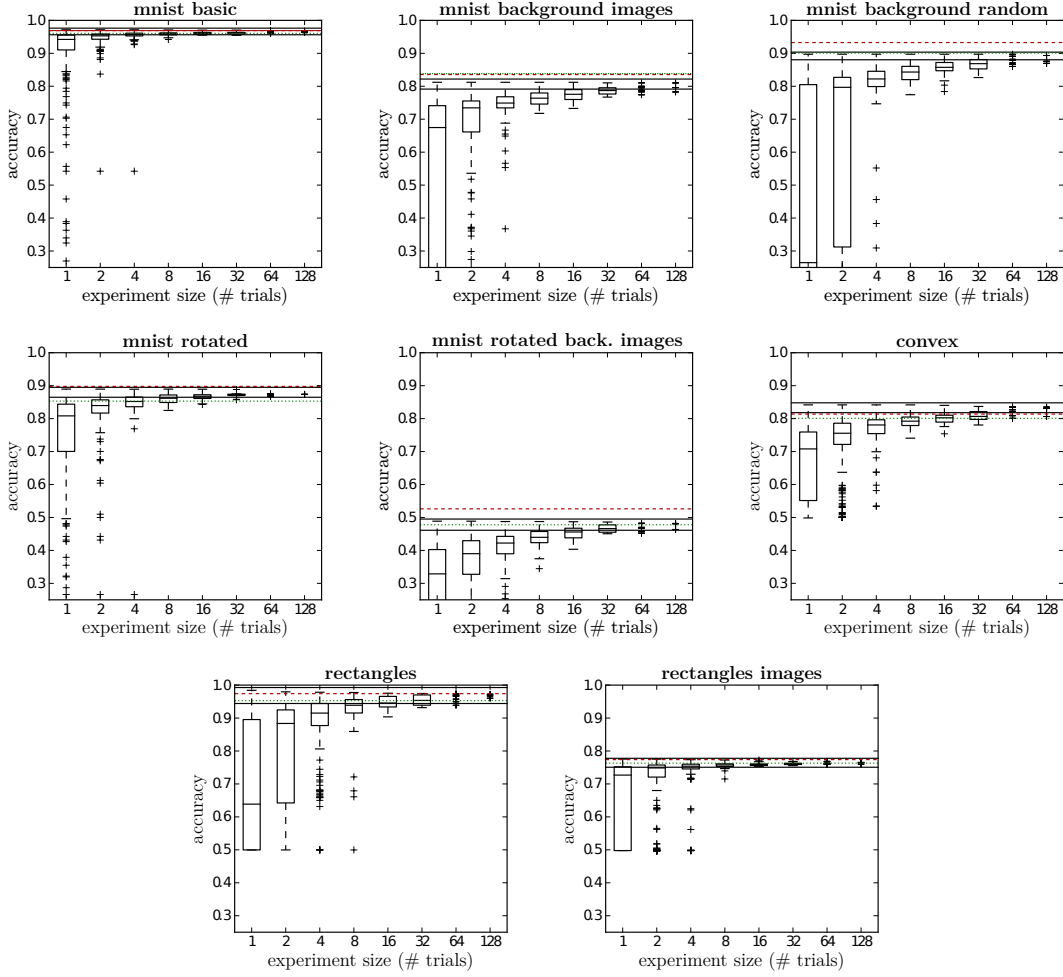


Figure 9: Deep Belief Network (DBN) performance according to random search. Here random search is used to explore up to 32 hyper-parameters. Results obtained by grid-assisted manual search using an average of 41 trials are marked in finely-dashed green (1-layer DBN) and coarsely-dashed red (3-layer DBN). Random experiments of 128 random trials found an inferior best model for three data sets, a competitive model in four, and superior model in one (**convex**). (*Best viewed in color.*)

of trials across data sets for the DBN-3 model was 41. In considering the number of trials per data set, it is important to bear in mind that the experiments on different data sets were not performed independently. Rather, later experiments benefited from the experience the authors had drawn from earlier ones. Although grid search was part of the optimization loop, the manual intervention turns the overall optimization process into something with more resemblance to an adaptive sequential algorithm.

Random search versions of the DBN experiments from Larochelle et al. (2007) are shown in Figure 9. In this more challenging optimization problem random search is still effective, but not

superior as it was as in the case of neural network optimization. Comparing to the 3-layer DBN results in Larochelle et al. (2007), random search found a better model than the manual search in one data set (**convex**), an equally good model in four (**mnist basic**, **mnist rotated**, **rectangles**, and **rectangles images**), and an inferior model in three (**mnist background images**, **mnist background random**, **mnist rotated background images**). Comparing to the 1-layer DBN results, random search of the 1-layer, 2-layer and 3-layer configuration space found at least a good a model in all cases. In comparing these scores, the reader should bear in mind that the scores in the original experiments were not computed using the same score-averaging technique that we described in Section 2.1, and our averaging technique is slightly biased toward underestimation. In the DBN efficiency curves we see that even experiments with larger numbers of trials (64 and larger) feature significant variability. This indicates that the regions of the search space with the best performance are small, and randomly chosen i.i.d. trials do not reliably find them.

6. Future Work

Our result on the multidimensional interval task, together with the GPR characterization of the shape of Ψ , together with the computational constraint that hyper-parameter searches only draw on a few hundred trials, all suggest that pseudo-random or quasi-random trials are optimal for non-adaptive hyper-parameter search. There is still work to be done for each model family, to establish how it should be parametrized for i.i.d. random search to be as reliable as possible, but the most promising and interesting direction for future work is certainly in adaptive algorithms.

There is a large body of literature on global optimization, a great deal of which bears on the application of hyper-parameter optimization. General numeric methods such as simplex optimization (Nelder and Mead, 1965), constrained optimization by linear approximation (Powell, 1994; Weise, 2009), finite difference stochastic approximation and simultaneous prediction stochastic approximation (Kleinman et al., 1999) could be useful, as well as methods for search in discrete spaces such as simulated annealing (Kirkpatrick et al., 1983) and evolutionary algorithms (Rechenberg, 1973; Hansen et al., 2003). Drew and de Mello (2006) have already proposed an optimization algorithm that identifies effective dimensions, for more efficient search. They present an algorithm that distinguishes between important and unimportant dimensions: a low-discrepancy point set is used to choose points in the important dimensions, and unimportant dimensions are “padded” with thinner coverage and cheaper samples. Their algorithm’s success hinges on the rapid and successful identification of important dimensions. Sequential model-based optimization methods and particularly Bayesian optimization methods are perhaps more promising because they offer principled approaches to weighting the importance of each dimension (Hutter, 2009; Hutter et al., 2011; Srinivasan and Ramakrishnan, 2011).

With so many sophisticated algorithms to draw on, it may seem strange that grid search is still widely used, and, with straight faces, we now suggest using random search instead. We believe the reason for this state of affairs is a technical one. Manual optimization followed by grid search is easy to implement: grid search requires very little code infrastructure beyond access to a cluster of computers. Random search is just as simple to carry out, uses the same tools, and fits in the same workflow. Adaptive search algorithms on the other hand require more code complexity. They require client-server architectures in which a master process keeps track of the trials that have completed, the trials that are in progress, the trials that were started but failed to complete. Some kind of shared database and inter-process communication mechanisms are required. Trials in an adaptive

experiment cannot be queued up all at once; the master process must be involved somehow in the scheduling and timing of jobs on the cluster. These technical hurdles are not easy to jump with the standard tools of the trade such as MATLAB or Python; significant software engineering is required. Until that engineering is done and adopted by a community of researchers, progress on the study of sophisticated hyper-parameter optimization algorithms will be slow.

7. Conclusion

Grid search experiments are common in the literature of empirical machine learning, where they are used to optimize the hyper-parameters of learning algorithms. It is also common to perform multi-stage, multi-resolution grid experiments that are more or less automated, because a grid experiment with a fine-enough resolution for optimization would be prohibitively expensive. We have shown that random experiments are more efficient than grid experiments for hyper-parameter optimization in the case of several learning algorithms on several data sets. Our analysis of the hyper-parameter response surface (Ψ) suggests that random experiments are more efficient because not all hyper-parameters are equally important to tune. Grid search experiments allocate too many trials to the exploration of dimensions that do not matter and suffer from poor coverage in dimensions that are important. Compared with the grid search experiments of Larochelle et al. (2007), random search found better models in most cases and required less computational time.

Random experiments are also easier to carry out than grid experiments for practical reasons related to the statistical independence of every trial.

- The experiment can be stopped any time and the trials form a complete experiment.
- If extra computers become available, new trials can be added to an experiment without having to adjust the grid and commit to a much larger experiment.
- Every trial can be carried out asynchronously.
- If the computer carrying out a trial fails for any reason, its trial can be either abandoned or restarted without jeopardizing the experiment.

Random search is not incompatible with a controlled experiment. To investigate the effect of one hyper-parameter of interest X , we recommend random search (instead of grid search) for optimizing over other hyper-parameters. Choose one set of random values for these remaining hyper-parameters and use that same set for each value of X .

Random experiments with large numbers of trials also bring attention to the question of how to measure test error of an experiment when many trials have some claim to being best. When using a relatively small validation set, the uncertainty involved in selecting the best model by cross-validation can be larger than the uncertainty in measuring the test set performance of any one model. It is important to take both of these sources of uncertainty into account when reporting the uncertainty around the best model found by a model search algorithm. This technique is useful to all experiments (including both random and grid) in which multiple models achieve approximately the best validation set performance.

Low-discrepancy sequences developed for QMC integration are also good alternatives to grid-based experiments. In low dimensions (e.g., 1-5) our simulated results suggest that they can hold some advantage over pseudo-random experiments in terms of search efficiency. However, the trials

of a low-discrepancy experiment are not i.i.d. which makes it inappropriate to analyze performance with the random efficiency curve. It is also more difficult in practice to conduct a quasi-random experiment because like a grid experiment, the omission of a single point can be more severe. Finally, when there are many hyper-parameter dimensions relative to the computational budget for the experiment, a low-discrepancy trial set is not expected to behave very differently from a pseudo-random one.

Finally, the hyper-parameter optimization strategies considered here are non-adaptive: they do not vary the course of the experiment by considering any results that are already available. Random search was not generally as good as the sequential combination of manual and grid search from an expert (Larochelle et al., 2007) in the case of the 32-dimensional search problem of DBN optimization, because the efficiency of sequential optimization overcame the inefficiency of the grid search employed at each step of the procedure. Future work should consider sequential, adaptive search/optimization algorithms in settings where many hyper-parameters of an expensive function must be optimized jointly and the effective dimensionality is high. We hope that future work in that direction will consider random search of the form studied here as a baseline for performance, rather than grid search.

Acknowledgments

This work was supported by the National Science and Engineering Research Council of Canada and Compute Canada, and implemented with Theano (Bergstra et al., 2010).

References

- I. A. Antonov and V. M. Saleev. An economic method of computing LP_t -sequences. *USSR Computational Mathematics and Mathematical Physics*, 19(1):252–256, 1979.
- R. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, New Jersey, 1961.
- Y. Bengio. Learning deep architectures for AI. *Foundations and Trends in Machine Learning*, 2(1): 1–127, 2009. doi: 10.1561/2200000006.
- Y. Bengio and X. Glorot. Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh and M. Titterton, editors, *Proc. of The Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS’10)*, pages 249–256, 2010.
- J. Bergstra, O. Breuleux, F. Bastien, P. Lamblin, R. Pascanu, G. Desjardins, J. Turian, and Y. Bengio. Theano: a CPU and GPU math expression compiler. In *Proceedings of the Python for Scientific Computing Conference (SciPy)*, June 2010. Oral.
- C. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, London, UK, 1995.
- P. Bratley, B. L. Fox, and H. Niederreiter. Implementation and tests of low-discrepancy sequences. *Transactions on Modeling and Computer Simulation, (TOMACS)*, 2(3):195–213, 1992.
- R. E. Caflisch, W. Morokoff, and A. Owen. Valuation of mortgage backed securities using brownian bridges to reduce effective dimension, 1997.

- C. Chang and C. Lin. *LIBSVM: A Library for Support Vector Machines*, 2001.
- I. Czogiel, K. Luebke, and C. Weihs. Response surface methodology for optimizing hyper parameters. Technical report, Universität Dortmund Fachbereich Statistik, September 2005.
- S. S. Drew and T. Homem de Mello. Quasi-Monte Carlo strategies for stochastic optimization. In *Proc. of the 38th Conference on Winter Simulation*, pages 774 – 782, 2006.
- D. Erhan, Y. Bengio, A. Courville, P. Manzagol, P. Vincent, and S. Bengio. Why does unsupervised pre-training help deep learning? *Journal of Machine Learning Research*, 11:625–660, 2010.
- J. H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 2:84–90, 1960.
- N. Hansen, S. D. Müller, and P. Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary Computation*, 11(1):1–18, 2003.
- G. E. Hinton. A practical guide to training restricted Boltzmann machines. Technical Report 2010-003, University of Toronto, 2010. version 1.
- G. E. Hinton, S. Osindero, and Y. Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18:1527–1554, 2006.
- F. Hutter. *Automated Configuration of Algorithms for Solving Hard Computational Problems*. PhD thesis, University of British Columbia, 2009.
- F. Hutter, H. Hoos, and K. Leyton-Brown. Sequential model-based optimization for general algorithm configuration. In *LION-5*, 2011. Extended version as UBC Tech report TR-2010-10.
- S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- N. L. Kleinman, J. C. Spall, and D. Q. Naiman. Simulation-based optimization with stochastic approximation using common random numbers. *Management Science*, 45(11):1570–1578, November 1999. doi: doi:10.1287/mnsc.45.11.1570.
- H. Larochelle, D. Erhan, A. Courville, J. Bergstra, and Y. Bengio. An empirical evaluation of deep architectures on problems with many factors of variation. In Z. Ghahramani, editor, *Proceedings of the Twenty-fourth International Conference on Machine Learning (ICML’07)*, pages 473–480. ACM, 2007.
- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998a.
- Y. LeCun, L. Bottou, G. Orr, and K. Muller. Efficient backprop. In G. Orr and K. Muller, editors, *Neural Networks: Tricks of the Trade*. Springer, 1998b.
- M. Galassi et al. *GNU Scientific Library Reference Manual*, 3rd edition, 2009.

- M. D. McKay, R. J. Beckman, and W. J. Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 21(2): 239–245, May 1979. doi: doi:10.2307/1268522.
- A. Nareyek. Choosing search heuristics by non-stationary reinforcement learning. *Applied Optimization*, 86:523–544, 2003.
- R. M. Neal. Assessing relevance determination methods using DELVE. In C. M. Bishop, editor, *Neural Networks and Machine Learning*, pages 97–129. Springer-Verlag, 1998.
- J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7: 308–313, 1965.
- M. J. D. Powell. A direct search optimization method that models the objective and constraint functions by linear interpolation. *Advances in Optimization and Numerical Analysis*, pages 51–67, 1994.
- C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- Ingo Rechenberg. *Evolutionsstrategie - Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Fommann-Holzboog, Stuttgart, 1973.
- A. Srinivasan and G. Ramakrishnan. Parameter screening and optimisation for ILP using designed experiments. *Journal of Machine Learning Research*, 12:627–662, February 2011.
- P. Vincent, H. Larochelle, Y. Bengio, and P. Manzagol. Extracting and composing robust features with denoising autoencoders. In W. W. Cohen, A. McCallum, and S. T. Roweis, editors, *Proceedings of the Twenty-fifth International Conference on Machine Learning (ICML’08)*, pages 1096–1103. ACM, 2008.
- T. Weise. *Global Optimization Algorithms - Theory and Application*. Self-Published, second edition, 2009. Online available at <http://www.it-weise.de/>.

DEEP LEARNING FOR FINANCE

MODULE 4 | LESSON 2

Implementing univariate Recurrent Neural Networks (RNN)

Reading Time	40 minutes
---------------------	------------

Prior Knowledge	RNN, Market timing, Deep Neural Networks, Dropout
------------------------	---

Keywords	RNN, Market timing
-----------------	--------------------

1. Introduction and data gathering

In our first practical contact with Recurrent Neural Networks (RNNs) we will implement a model to predict stock **prices** (rather than returns, as we have seen in previous modules). This has an importance, because prices are closer to be interpreted as a sequences than returns. This makes them potentially more predictive using RNNs than returns.

As usual, we will start by downloading price data from Yahoo! finance. Note that we take prices from just one stock, our usual suspect Apple Inc., to explore the time-series univariate prediction case. In lesson 4 we will extend this to a multivariate case.

```
In [1]: import numpy as np
import pandas as pd
import yfinance as yf
from sklearn.preprocessing import MinMaxScaler
```

We start by gathering the data and defining the output as next day's close price:

```
In [2]: # Read the CSV file
file_path = 'AAPL.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=['Date'], da
df = df.sort_values('Date')
df = df.set_index('Date')

df['Output'] = df.Price.shift(-1)
# Reset the index to make Date a regular column
df = df.reset_index(names=['Date'])
# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()

df.head()
```

Out[2]:

	Date	Price	CVol	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open
2773	1991-12-02	0.46	118899320.0	0.01	1.97	1.97	0.48	276,80	0.45
2774	1991-12-03	0.45	102860910.0	-0.01	-2.42	-2.42	0.47	267,70	0.46
2776	1991-12-05	0.45	99198490.0	0.00	-0.99	-0.99	0.47	264,06	0.45
2777	1991-12-06	0.44	196985780.0	-0.01	-2.50	-2.50	0.46	254,96	0.44
2778	1991-12-09	0.44	97832080.0	0.00	0.77	0.77	0.46	257,69	0.44

In [3]:

```
# Convert 'Date' column to datetime for proper filtering
df['Date'] = pd.to_datetime(df['Date'])

# Filter for dates from 2010 onwards
df = df[df['Date'] >= '2010-01-01'].reset_index(drop=True)

print("Length of data frame: ", len(df))
df.head()
```

Length of data frame: 3852

Out[3]:

	Date	Price	CVol	Change	% Change	% Return	Total Return (Gross)	Cumulative Return %	Open	High
0	2010-01-04	7.64	493728720.0	0.12	1.56	1.56	8.37	6.427,19	7.62	7.64
1	2010-01-05	7.66	601904560.0	0.01	0.17	0.17	8.39	6.438,47	7.66	7.66
2	2010-01-06	7.53	552158800.0	-0.12	-1.59	-1.59	8.26	6.334,47	7.66	7.66
3	2010-01-07	7.52	477129720.0	-0.01	-0.18	-0.18	8.24	6.322,57	7.56	7.56
4	2010-01-08	7.57	447876700.0	0.05	0.66	0.66	8.29	6.365,27	7.51	7.57

2. Validation, train and test sample definition

Next, we proceed to define our training, test, and validation sets. Note that we will train using the 98% of the sample. The validation set will be 20% of the training test. We also need to define a 'window' of observations that we are going to take as a sequence input to our network. In this case we choose a window of 20, which means that the last 20 prices (observations) will be incorporated into the sequence of inputs for the network.

Note that not all of these may end up playing a big role in the prediction (this is for the training to decide), but we will give the network the possibility to consider up to 20 past prices.

Also, note that these, and others, parameters can be included as hyperparameters of the models that we can tune to achieve maximum performance of the algorithm. This is something we leave for future implementations of the Neural Network.

```
In [4]: # First, we deal with the definition of the training, test, and validations sets
val_split = 0.2
train_split = 0.98
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 20 # How many daily prices do I want to incorporate in the input

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

# Second, we actually define the input matrix for train (X_train) and test (X_test)
# Note that these will need further work afterwards, in order to incorporate the
Xdf, ydf = df.iloc[:, 1:2], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]

# Third, we proceed with scaling inputs to the model. Note how this is specially
scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)

# Lastly, because we want a time series with up to 20 (window_size) past observations
training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
```

```

y_train.append(y_train_set_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)

print("Size of X vector in training:", X_train.shape)
print("Size of Y vector in training:", y_train.shape)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)
print("Size of X vector in test:", X_test.shape)
print("Size of Y vector in test:", y_test.shape)
print("Number of features in the model: ", n_features)

```

```

Size of X vector in training: (3754, 20, 1)
Size of Y vector in training: (3754, 1)
Size of X vector in test: (58, 20, 1)
Size of Y vector in test: (78, 1)
Number of features in the model: 1

```

3. Defining the RNN model

Once we have pre-processed the input and output data of our network, let's build it!

We will start with a very simple RNN that considers 5 hidden layers (4 simple RNN layers + 1 final dense layer). Each of the RNN layers will have 50 neurons. We apply a Dropout of 20% after each of the RNN layers. Finally, before the single-unit final output layer, we will apply a dense layer with 10 neurons. As you know already, this architecture is kind of random. We will play around with the composition of the network once we move to more advanced models through hyperparameter tuning.

A novel thing of RNNs is that we can set the 'return_sequences' argument to True or False, depending on whether we want to pass the last output (if False) or the whole sequence (if True) to the next layer. More info on this here:

https://www.tensorflow.org/api_docs/python/tf/keras/layers/SimpleRNN

```

In [5]: import tensorflow as tf

model = tf.keras.models.Sequential(
    [
        tf.keras.layers.SimpleRNN(
            50, return_sequences=True, input_shape=(X_train.shape[1], n_features
        ),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=False),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(10),
    ]
)

```

```

        tf.keras.layers.Dense(1),
    ]
)

```

2025-05-14 13:05:07.100468: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-14 13:05:07.148245: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-14 13:05:08.476403: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

/usr/local/lib/python3.11/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

    super().__init__(**kwargs)

```

So, how does the model architecture look like?

In [6]: `model.summary()`

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 50)	2,600
dropout (Dropout)	(None, 20, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 50)	5,050
dropout_1 (Dropout)	(None, 20, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 20, 50)	5,050
dropout_2 (Dropout)	(None, 20, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5,050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 10)	510
dense_1 (Dense)	(None, 1)	11

Total params: 18,271 (71.37 KB)

Trainable params: 18,271 (71.37 KB)

Non-trainable params: 0 (0.00 B)

Note how here the number of parameters to be trained by the model is defined a little bit different at each step. For example, for the second RNN layer, note that we have a total of 5,050 parameters. Where does this number come from?

Well, $50 \times 50 + 50 \times 50 + 50 \times 1 = 5,050$ parameters. In other words, outputs from previous layer (50) times neurons in this layer (50) plus the neurons in the layer times themselves (50×50), plus the bias term vector (50×1).

Where does the 2,600 parameters of the first layer come from? Well, 50×50 (neurons in the layer) + 50×1 (neurons times number of model features) + 50×1 (bias terms) = 2,600

Alike we did for other models, if we sum up all the parameters in the different layers we end up with 18,271 parameters to be trained in this basic model.

4. Model training

Now, we can proceed with model training. As you will see, there is nothing really new about this and basically training works in exactly the same way once we have defined our hidden layers as 'recurrent' ones.

```
In [7]: from keras.callbacks import EarlyStopping

hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr), loss="mean_absolute
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=1, patience=10, restore_best_weights
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=500,
    batch_size=64,
    verbose=1,
    callbacks=[es],
)
```

Epoch 1/500			
47/47		5s 27ms/step	loss: 0.8755 - val_loss: 0.5018
Epoch 2/500			
47/47		2s 15ms/step	loss: 0.7682 - val_loss: 0.3504
Epoch 3/500			
47/47		1s 15ms/step	loss: 0.6676 - val_loss: 0.2727
Epoch 4/500			
47/47		1s 17ms/step	loss: 0.6242 - val_loss: 0.2808
Epoch 5/500			
47/47		1s 15ms/step	loss: 0.5754 - val_loss: 0.1098
Epoch 6/500			
47/47		1s 15ms/step	loss: 0.5630 - val_loss: 0.1120
Epoch 7/500			
47/47		1s 15ms/step	loss: 0.5124 - val_loss: 0.1234
Epoch 8/500			
47/47		1s 15ms/step	loss: 0.4827 - val_loss: 0.1160
Epoch 9/500			
47/47		1s 15ms/step	loss: 0.4461 - val_loss: 0.0858
Epoch 10/500			
47/47		1s 15ms/step	loss: 0.4200 - val_loss: 0.1336
Epoch 11/500			
47/47		1s 15ms/step	loss: 0.3870 - val_loss: 0.0733
Epoch 12/500			
47/47		1s 15ms/step	loss: 0.3677 - val_loss: 0.0799
Epoch 13/500			
47/47		1s 15ms/step	loss: 0.3573 - val_loss: 0.0771
Epoch 14/500			
47/47		1s 16ms/step	loss: 0.3136 - val_loss: 0.1388
Epoch 15/500			
47/47		1s 15ms/step	loss: 0.3070 - val_loss: 0.0667
Epoch 16/500			
47/47		1s 15ms/step	loss: 0.2891 - val_loss: 0.2095
Epoch 17/500			
47/47		1s 15ms/step	loss: 0.2694 - val_loss: 0.0884
Epoch 18/500			
47/47		1s 15ms/step	loss: 0.2530 - val_loss: 0.0768
Epoch 19/500			
47/47		1s 15ms/step	loss: 0.2293 - val_loss: 0.1024
Epoch 20/500			
47/47		1s 15ms/step	loss: 0.2189 - val_loss: 0.0873
Epoch 21/500			
47/47		1s 15ms/step	loss: 0.1981 - val_loss: 0.0665
Epoch 22/500			
47/47		1s 15ms/step	loss: 0.1845 - val_loss: 0.0676
Epoch 23/500			
47/47		1s 15ms/step	loss: 0.1743 - val_loss: 0.0637
Epoch 24/500			
47/47		1s 15ms/step	loss: 0.1633 - val_loss: 0.0608
Epoch 25/500			
47/47		1s 15ms/step	loss: 0.1587 - val_loss: 0.0647
Epoch 26/500			
47/47		1s 15ms/step	loss: 0.1468 - val_loss: 0.0677
Epoch 27/500			
47/47		1s 15ms/step	loss: 0.1421 - val_loss: 0.0674
Epoch 28/500			
47/47		1s 15ms/step	loss: 0.1300 - val_loss: 0.0644
Epoch 29/500			
47/47		1s 15ms/step	loss: 0.1202 - val_loss: 0.0639
Epoch 30/500			
47/47		1s 15ms/step	loss: 0.1203 - val_loss: 0.0648

```
Epoch 31/500
47/47 ————— 1s 15ms/step - loss: 0.1103 - val_loss: 0.0630
Epoch 32/500
47/47 ————— 1s 15ms/step - loss: 0.1074 - val_loss: 0.0618
Epoch 33/500
47/47 ————— 1s 15ms/step - loss: 0.1037 - val_loss: 0.0761
Epoch 34/500
47/47 ————— 1s 16ms/step - loss: 0.0978 - val_loss: 0.0626
Epoch 34: early stopping
Restoring model weights from the end of the best epoch: 24.
```

```
Out[7]: <keras.src.callbacks.history.History at 0x714a7b4dfe90>
```

Next, as usual, we will visualize our predictions in the test sample, undergoing the usual inverse transform to un-do the scaling process:

```
In [8]: prediction = model.predict(X_test)
prediction = scaler_output.inverse_transform(prediction)
prediction = prediction.flatten()

values = np.array(y_test[window_size:])
values = values.flatten()
```

```
2/2 ————— 1s 310ms/step
```

```
In [9]: df_predictions = pd.DataFrame(
        {"Date": test_time.flatten(), "Pred RNN": prediction, "values": values}
    )
df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%Y-%m-%d")
df_predictions.head()
```

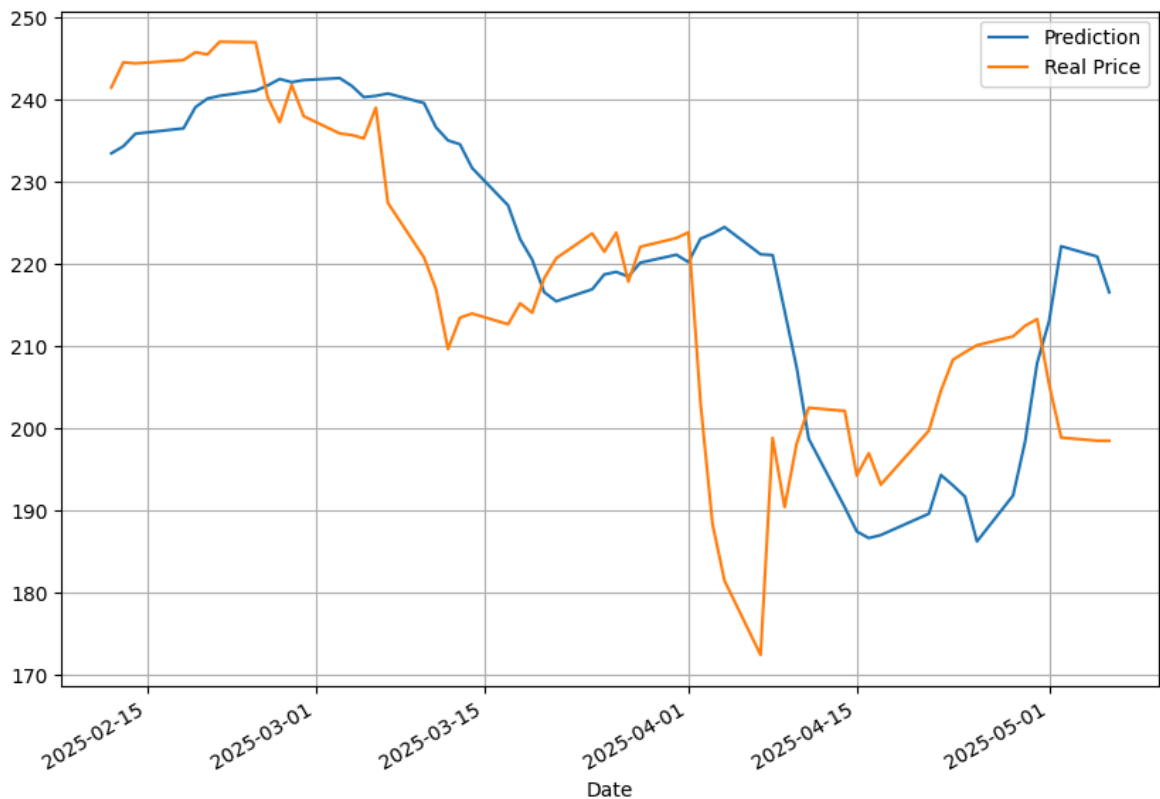
```
Out[9]:
```

	Date	Pred RNN	values
0	2025-02-12	233.512192	241.529999
1	2025-02-13	234.372055	244.600006
2	2025-02-14	235.897598	244.470001
3	2025-02-18	236.552780	244.869995
4	2025-02-19	239.142578	245.830002

And, finally, in a plot:

```
In [10]: import matplotlib.pyplot as plt

plt_1 = plt.figure(figsize=(10, 7))
ax = plt.gca()
df_predictions.plot(x="Date", y="Pred RNN", label="Prediction", ax=ax)
df_predictions.plot(x="Date", y="values", label="Real Price", ax=ax)
plt.grid()
plt.show()
```



As you can see, our model does not perform so well for predicting the time-series of prices given a sequence. It kind of 'guesses' part of the trend, but is far from being a nice tool to closely predict future stock prices.

5. Conclusion

In this lesson we have implemented our first Recurrent Neural Network (RNN) using a sequence of 20 past prices to predict future ones. We have seen that the predictive performance of this model is not so great after all. But there are lots of things to be done in terms of improving the model. This is what we will tackle in the next lessons, which will also serve as a good excuse to introduce the details of how forward and backprop work in RNN models.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

DEEP LEARNING FOR FINANCE

MODULE 4 | LESSON 4

Multivariate RNNs

Reading Time	45 minutes
Prior Knowledge	RNN, Deep Neural Network, Market timing
Keywords	Multivariate RNN, VIX, S&P500

1. Introduction: S&P500 index and the VIX

It is now time to augment our deep RNNs to consider more than one series of inputs. In this lesson we revisit the market timing example of Lesson 2, but we will now predict stock prices using past information (prices) about the security as well as a leading indicator variable. Specifically, we will try to predict the prices of an Exchange Traded Fund (ETF) that replicates the composition and behavior of the S&P500 index in the U.S., the famous 'SPY'. And we will use for that not only past prices of SPY, but also the prices (levels) of a leading indicator of S&P500 index: the VIX index.

The VIX is a volatility index of the S&P500 market index. It is constructed using the implied volatility from a portfolio of 30-day constant-maturity options on S&P500 index. Since we all well know at this point, the fact that VIX index is constructed using options-implied volatility information means that it reflects investor's expectations on the underlying asset, the S&P500 index (Ticker symbol 'SPX'). More information on the VIX, and how it can be useful in the next few bullet points (please note these materials are only informative, not investment recommendation!):

- More information about the VIX index by the 'creators', Chicago Board Options Exchange (CBOE): https://www.cboe.com/tradable_products/vix/faqs/
- How investors & traders use the VIX: <https://optionshawk.com/how-to-use-the-vix-as-a-market-signal/>
- A web were you can consult the 'term structure' of Future contracts over VIX with different maturities: <http://vixcentral.com/>
- How to interpret the VIX term structure: <https://www.projectfinance.com/vix-term-structure/>
- An interesting paper on how to use VIX term structure to predict recessions: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3943982

- A great academic paper on how VIX term structure predicts risk premia:
<https://www.cambridge.org/core/services/aop-cambridge-core/content/view/56572D1F060448571BD8F597C732D9C3/S0022109017000825a.pdf/r>
- On using VIX in investing under a formal, systematic approach: <https://joi.pm-research.com/content/23/3/130.short>

When you come to think about it, if VIX really works as a powerful leading indicator of SPX, adding it as an input to our RNN prediction problem should be like 'cheating'. If we think about another type of prediction task suited for RNNs, such as language models, it would be like asking the model to predict how a sentence would end knowing what are the next few words. It should simplify the prediction task a lot!

Spoiler alert! It will not. Or, at least, it will not necessarily do it. For example, what if investors already looked at VIX prices to decide on their trading in the S&P500? In that case, it is likely that adding the VIX to our input matrix set could be even redundant, as this information is already incorporated into prices themselves.

Of course, it may be the case that investors (or at least some of them) are oblivious to information contained in VIX. For starters, it is likely that investors trading the S&P500 index and its components are inherently different from those trading options on the SPX. These differences can arise not only in terms of how informed are each of these guys, but also their investment horizon, risk-aversion, ... All in all, these features (and others, like limits to arbitrage) cast doubts (at least ex-ante) on whether S&P500 incorporates data on VIX and the potential predictive power of VIX as a leading indicator.

So... let's resolve this question!

2. Data gathering

As usual, we will start with the data gathering process. Note how in this case we do not only gather daily stock prices from our S&P500 index ETF, SPY, from Yahoo! finance, but also on the daily prices (levels) of VIX. We will be working, as usual, with daily prices and trying to predict what is the next price that SPY will take.

```
In [1]: import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Read the CSV file
file_path = 'SPY.csv'
spy_df = pd.read_csv('SPY.csv', sep=';', decimal=',')
spy_df['Date'] = pd.to_datetime(spy_df['Date'], format='%m/%d/%y')
# Create a column for next day's SPY price (Output)
spy_df['Output'] = spy_df['Price'].shift(-1)

spy_df = spy_df.dropna()
```

```
spy_df.head()
```

Out[1]:

	Date	Price	CVol	% Change	Open	Low	High	NAV	Total Return (Gross)	% Return	Ci
1	2025-05-13	586.84	67947234	0.66	583.41	582.84	589.08	587.02	776.06	0.66	
2	2025-05-12	582.99	78993570	3.30	581.47	577.04	583.00	582.79	770.97	3.30	
3	2025-05-09	564.34	37603434	-0.13	566.48	562.76	567.50	564.37	746.30	-0.13	
4	2025-05-08	565.06	65130793	0.70	565.24	561.70	570.31	564.68	747.26	0.70	
5	2025-05-07	561.15	55733230	0.42	560.15	556.04	563.82	561.42	742.09	0.42	

```
In [2]: vix_df = pd.read_csv('VIX.csv', sep=';', decimal=',')
vix_df['Date'] = pd.to_datetime(vix_df['Date'], format='%m/%d/%y')
vix_df.head()
```

Out[2]:

	Date	Price	CVol	Change	% Change	Total Return (Gross, Unhedged)	Cumulative Return % (Gross, Unhedged)	Open	High	Low
0	2025-05-14	18.22	NaN	NaN	NaN	NaN	NaN	NaN	18.22	18.22
1	2025-05-13	18.22	NaN	-0.17	-0.92	NaN	NaN	18.69	18.85	17.65
2	2025-05-12	18.39	NaN	-3.51	-16.03	NaN	NaN	19.84	20.44	18.14
3	2025-05-09	21.90	NaN	-0.58	-2.58	NaN	NaN	22.40	22.82	21.83
4	2025-05-08	22.48	NaN	-1.07	-4.54	NaN	NaN	22.84	23.61	21.88

```
In [3]: # Merge the two dataframes on Date
df = pd.merge(
    spy_df[['Date', 'Price', 'Output']],
    vix_df[['Date', 'Price']],
    on='Date',
    how='inner'
)

# Rename columns for clarity
df = df.rename(columns={
    'Price_x': 'SPY_Price',
    'Price_y': 'VIX_Price'
})
```

```

# Drop any rows with NaN values (will be the last row due to shift operation)
df = df.dropna()

# Sort by date (newest first, matching the original data order)
df = df.sort_values('Date', ascending=False)

# Reset index
df = df.reset_index(drop=True)

# Reorder columns to make Output the last column
df = df[['Date', 'VIX_Price', 'SPY_Price', 'Output']]

# Display the first few rows of the final dataframe
df.head()

```

Out[3]:

	Date	VIX_Price	SPY_Price	Output
0	2025-05-13	18.22	586.84	582.99
1	2025-05-12	18.39	582.99	564.34
2	2025-05-09	21.90	564.34	565.06
3	2025-05-08	22.48	565.06	561.15
4	2025-05-07	23.55	561.15	558.80

3. Validation, train and test sample definition

Next, we proceed to define our training, test, and validation sets. Note that we will train using the 98% of the sample. The validation set will be 20% of the training test. We also need to define a 'window' of observations that we are going to take as a sequence input to our network. In this case we choose a window of 20, which means that the last 20 prices (observations) will be incorporated into the sequence of inputs for the network. In this case this means that we will have a sequence of 20 SPY prices, as well as 20 VIX prices that will be part of the training at each t .

```

In [5]: # First, we deal with the definition of the training, test, and validations sets
val_split = 0.2
train_split = 0.98
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 20 # How many daily prices do I want to incorporate in the input

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

# Second, we actually define the input matrix for train (X_train) and test (X_test)
Xdf, ydf = (
    df.iloc[:, 1:3],
    df.iloc[:, -1],
) # Note how we choose close prices from both SPY and VIX in the X matrix

```

```

X = Xdf.astype("float32")
y = ydf.astype("float32")

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]

# Third, we proceed with scaling inputs to the model.
scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)

## Lastly, because we want a time series with up to 20 (window_size) past obser
training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
    y_train.append(y_train_set_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)

print(X_train.shape)
print(y_train.shape)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)
print(X_test.shape)
print(y_test.shape)
print(n_features)

```

(3757, 20, 2)

(3757, 1)

(58, 20, 2)

(78, 1)

2

4. Defining the RNN model

Once we have pre-processed the input and output data of our network, let's build it!

Our network architecture will be identical to the one from Lesson 2. We consider a simple RNN 5 hidden layers (4 simple RNN layers + 1 final dense layer). Each of the RNN layers will have 50 neurons. We apply a Dropout of 20% after each of the RNN layers. Finally, before the single-unit final output layer, we will apply a dense layer with 10 neurons.

As we already mentioned, this choice of architecture is as good as random. We leave to you to play around with the architecture and check how predictions vary.

Remember that a good resource to have nearby when implementing any Deep Neural Network is the TensorFlow/Keras documentation:

<https://www.tensorflow.org/guide/keras/rnn?hl=en>

```
In [6]: import tensorflow as tf

model = tf.keras.models.Sequential(
    [
        tf.keras.layers.SimpleRNN(
            50, return_sequences=True, input_shape=(X_train.shape[1], n_features
        ),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=True),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.SimpleRNN(50, return_sequences=False),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(10),
        tf.keras.layers.Dense(1),
    ]
)
```

2025-05-14 11:13:41.036553: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-05-14 11:13:41.077864: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

2025-05-14 11:13:42.152523: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

/usr/local/lib/python3.11/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Let's now check how our model looks like:

```
In [7]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 50)	2,650
dropout (Dropout)	(None, 20, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 50)	5,050
dropout_1 (Dropout)	(None, 20, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 20, 50)	5,050
dropout_2 (Dropout)	(None, 20, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5,050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 10)	510
dense_1 (Dense)	(None, 1)	11



Total params: 18,321 (71.57 KB)

Trainable params: 18,321 (71.57 KB)

Non-trainable params: 0 (0.00 B)

Now, the number of parameters in the first layer of the model changes a little bit. Where does the 2,650 come from?

Well, 50×50 (neurons in the layer) + 50×2 (neurons times number of model features) + 50×1 (bias terms) = 2,650

Note how the total model parameters is now 18,321. Only 50 more (because there is 1 more input feature) than the similar model from Lesson 2. Finally, note how the only impact on model parameters from incorporating 1 more feature comes in the first layer of the model. After that, there is no increase in the trainable parameters resulting from considering more input features. This is also important to take into account when considering how the 'size' of the model may impact the performance of backprop and gradient descent algorithm: It is much more about the 'length' of the input sequence than the number of input features themselves!

5. Model training

Now, we can proceed with model training.

```
In [8]: from keras.callbacks import EarlyStopping

hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr), loss="mean_absolute
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=1, patience=10, restore_best_weights
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=500,
    batch_size=64,
    verbose=1,
    callbacks=[es],
)
```

```

Epoch 1/500
47/47 ————— 4s 21ms/step - loss: 0.6665 - val_loss: 0.3481
Epoch 2/500
47/47 ————— 1s 13ms/step - loss: 0.5605 - val_loss: 0.1852
Epoch 3/500
47/47 ————— 1s 12ms/step - loss: 0.4981 - val_loss: 0.1422
Epoch 4/500
47/47 ————— 1s 13ms/step - loss: 0.4719 - val_loss: 0.1242
Epoch 5/500
47/47 ————— 1s 13ms/step - loss: 0.4521 - val_loss: 0.2171
Epoch 6/500
47/47 ————— 1s 13ms/step - loss: 0.4232 - val_loss: 0.1209
Epoch 7/500
47/47 ————— 1s 12ms/step - loss: 0.3865 - val_loss: 0.0979
Epoch 8/500
47/47 ————— 1s 13ms/step - loss: 0.3772 - val_loss: 0.1061
Epoch 9/500
47/47 ————— 1s 12ms/step - loss: 0.3500 - val_loss: 0.0619
Epoch 10/500
47/47 ————— 1s 14ms/step - loss: 0.3317 - val_loss: 0.1167
Epoch 11/500
47/47 ————— 1s 14ms/step - loss: 0.3162 - val_loss: 0.0567
Epoch 12/500
47/47 ————— 1s 12ms/step - loss: 0.3087 - val_loss: 0.1183
Epoch 13/500
47/47 ————— 1s 12ms/step - loss: 0.2763 - val_loss: 0.1875
Epoch 14/500
47/47 ————— 1s 12ms/step - loss: 0.2662 - val_loss: 0.0869
Epoch 15/500
47/47 ————— 1s 12ms/step - loss: 0.2599 - val_loss: 0.1384
Epoch 16/500
47/47 ————— 1s 13ms/step - loss: 0.2407 - val_loss: 0.1587
Epoch 17/500
47/47 ————— 1s 14ms/step - loss: 0.2301 - val_loss: 0.0672
Epoch 18/500
47/47 ————— 1s 12ms/step - loss: 0.2247 - val_loss: 0.0813
Epoch 19/500
47/47 ————— 1s 12ms/step - loss: 0.2118 - val_loss: 0.1060
Epoch 20/500
47/47 ————— 1s 12ms/step - loss: 0.2041 - val_loss: 0.0615
Epoch 21/500
47/47 ————— 1s 13ms/step - loss: 0.1937 - val_loss: 0.1089
Epoch 21: early stopping
Restoring model weights from the end of the best epoch: 11.

```

```
Out[8]: <keras.src.callbacks.history.History at 0x703649975850>
```

Then we can make our predictions on the test sample based on the trained model parameters:

```

In [9]: prediction = model.predict(X_test)
        prediction = scaler_output.inverse_transform(prediction)
        prediction = prediction.flatten()

        values = np.array(y_test[window_size:])
        values = values.flatten()

```

```
2/2 ————— 1s 259ms/step
```

```
In [10]: df_predictions = pd.DataFrame(
    {"Date": test_time.flatten(), "Pred RNN": prediction, "values": values}
)
df_predictions.Date = pd.to_datetime(df_predictions.Date, format="%Y-%m-%d")
df_predictions.head()
```

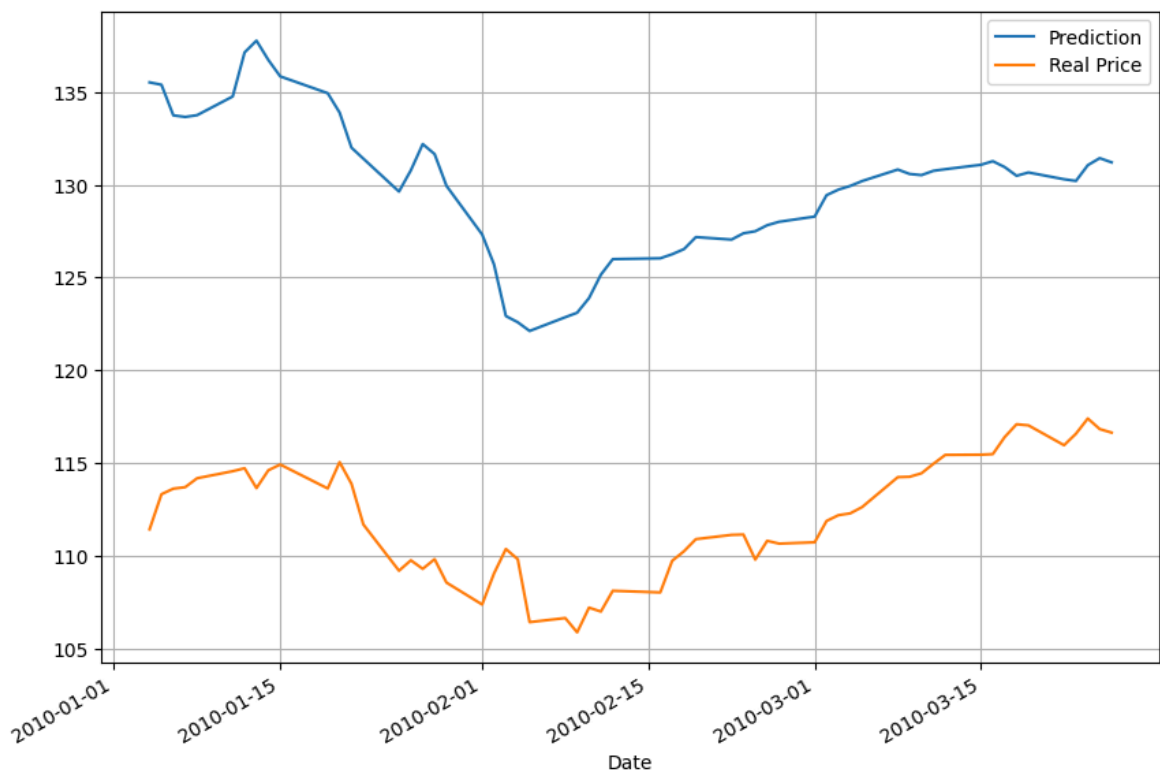
```
Out[10]:
```

	Date	Pred RNN	values
0	2010-03-26	131.208176	116.650002
1	2010-03-25	131.434296	116.839996
2	2010-03-24	131.056564	117.410004
3	2010-03-23	130.207977	116.589996
4	2010-03-22	130.297607	115.970001

And visualize it,

```
In [11]: import matplotlib.pyplot as plt

plt_1 = plt.figure(figsize=(10, 7))
ax = plt.gca()
df_predictions.plot(x="Date", y="Pred RNN", label="Prediction", ax=ax)
df_predictions.plot(x="Date", y="values", label="Real Price", ax=ax)
plt.grid()
plt.show()
```



As you can see (and please remember that we have not fixed the random seed so you may obtain different outcomes every time you train the model), the VIX does not really improve the predictive performance of the model by a lot (sometimes it's even making it worse!).

Can you think of ways to augment and improve this model? Well, one option may be using other types of 'leading variables'. For example, some variables used in practice for this are: prices of oil futures, manufacturing PMI in the U.S., agricultural production, ...

But we would usually not like a model that uses too many inputs because it will probably lead not only to increase training time and computational resources, but also to a lower degree of understanding regarding the interplay between the different features. Bottom line, it is important to understand the basics of how models work before we start adding inputs!

6. A final note on NLP

One of the main uses of RNNs is Natural Language Processing (NLP), which is a fascinating area. Sadly, there are, yet, not so many applications of NLP to financial market problems, which is why we have not focused on language models in this module.

In the book by Zhang. et. al. (2022) that we have been using as a reference for the whole course you have much more information on NLP models in case you are interested. Specifically, Chapters 15 and 16 deal with the pre-training and direct applications of NLP models in practice. Note that RNNs are just one part of NLP models, but probably the ones that make more sense when focusing on areas such as speech recognition.

Here you can have more info for the use of NLP in finance:

- A few examples of finance-related applications of NLP:
<https://softengi.com/blog/nlp-use-cases-in-finance/>
- One important application of NLP in finance is sentiment analysis. Here you have a quick review of it: <https://research.aimultiple.com/sentiment-analysis-stock-market/>
- A paper by Mittal and Goel (Stanford) that uses Twitter sentiment analysis for stock price prediction: <http://cs229.stanford.edu/proj2011/GoelMittal-StockMarketPredictionUsingTwitterSentimentAnalysis.pdf>

These are just some examples to 'whet your appetite'. Surely you can find many more!

7. Conclusion

In this lesson, we have dealt with multivariate RNNs. This means that we have used several input features into a recursive neural network model. So far, we have not obtained great results, though. But we have used a very tiny portion of all the available tools in deep learning and recursive networks.

In the next module, we will explore more specific types of RNN models: Gated Recurrent Units (GRUs) and Long Short-Term Memory (LSTM). See you in the next module!

Module 4: Lesson 1

Recurrent Neural Networks (RNNs)



Outline

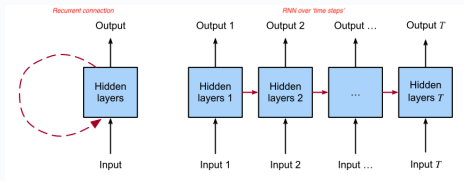
- ▶ What are RNNs and why are they useful?
- ▶ Working with sequences
- ▶ Latent auto-regressive models
- ▶ Recurrent Neural Networks

What are RNNs and why are they useful?

We already know how to build neural networks that work with data of different lengths and characteristics such as images, but we haven't dealt with sequential data yet:

- What should we do when the order of the input data matters for the prediction?
 - Sequence of images → Video!
 - Sequence of words → Text! → Natural Language Processing (NLP)

Recurrent Neural Networks (RNNs) are deep learning models that capture the dynamics of sequences via recurrent connections—i.e., cycles in the network.



Working with sequences

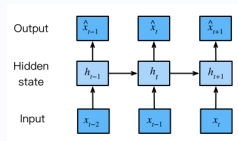
The fact that a specific sequence of input data matters for the prediction of the next is very intuitive:

- what is behind the auto-complete feature of your phones when you text?

However, working with sequences in a deep neural network has some challenges. In order to properly understand what RNNs do, let's start from a baseline: **autoregressive (AR) models**.

- ▶ AR models aim to predict a future value x_t given the past values of this variable: $\mathbb{E}[x_t | x_{t-1}, \dots, x_{t-n}]$
- ▶ Problem #1: How many past observations do we need? → in Latent AR models h_t summarizes the past

$$\hat{x}_t = \mathbb{P}(x_t | h_t), \text{ where } h_t = g(h_{t-1}, x_{t-1})$$

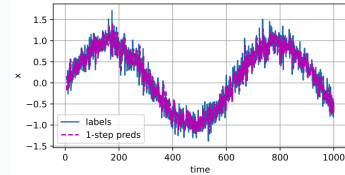
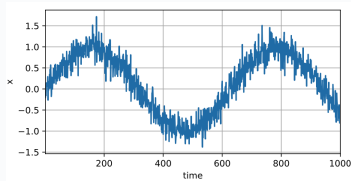


- ▶ Problem #2: Markov condition? → Given recent history, future conditionally independent of the past?

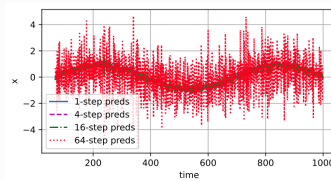
$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t | x_{t-1}, \dots, x_1)$$

Working with sequences

Suppose we want to predict the next logical step of a sin function with some noise like the one on the left. Then, we can use linear regression to predict the one-step-ahead values:



How about predicting 4 steps ahead? and 16? 64?



RNNs for NLP

One of the main uses of RNNs nowadays is in natural language processing (NLP). Unfortunately, the scope for applying NLP into quantitative finance problems is, for now, very limited.

We will not see in depth the use of RNNs for NLP during this course, but please note that the functioning of the whole network model that we will learn will be equivalent to that for NLP.

There is just one thing within the NLP framework that will need to be taken care of: **tokenization**:

- ▶ Tokens are the atomic (indivisible) units of text.
- ▶ A token can be entire words, characters, or even sentences.
- ▶ Still, tokens are simply strings that need to be integrated within a **vocabulary** in numerical form
- ▶ Once we have transformed text into sequence data we can simply train language models as estimates of the joint probability of a text sequence!

You can learn more about this in sections 9.2 and 9.3 in chapter 9 of Zhang et al.'s book *Dive into Deep Learning*.

Recurrent Neural Networks (RNNs)

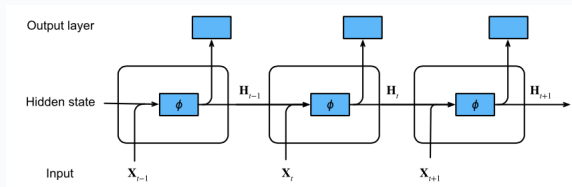
RNNs are neural networks **with hidden states**, similar to the latent AR model.

- First, let's revisit briefly how an NN **without hidden states** (e.g., MLP) worked:

$$H = \phi(XW_{xh} + b_h) \rightarrow O = HW_{hq} + b_q$$

- In an NN **with hidden states**, we save the hidden layer output (H_{t-1}) with a weight (W_{hh}) to account for the sequence. So, for time step t :

$$H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h) \rightarrow O_t = H_t W_{hq} + b_q$$



Summary of Lesson 1

In Lesson 1, we have:

- ▶ Introduced RNN
- ▶ Discussed models with hidden states
- ▶ Basics of building a RNN

⇒ **References for this lesson:**

Chapter 9, Sections 9.1 to 9.4 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice by building our first recurrent neural network.

Module 4: Lesson 2

RNN implementation



Outline

- ▶ Implementing an RNN
- ▶ Predicting stock prices with RNNs

Implementing a RNN in practice

In lesson 2, we will implement an RNN in practice for the first time. As you will see in the accompanying notebook, most of the process (at least from a coding perspective) is identical to what we have done before for other neural network architectures:

1. Gather data
2. Define samples: validation, train, and test
→ Note here that we will also need to define the 'window' for the sequence of inputs we want to consider!
3. Defining the model architecture

```
model = tf.keras.models.Sequential([
    tf.keras.layers.SimpleRNN(50, return_sequences=True, input_shape=(X_train.shape[1], n_features)),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=True),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.SimpleRNN(50, return_sequences=False),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10),
    tf.keras.layers.Dense(1)
])
```

4. Training the model!

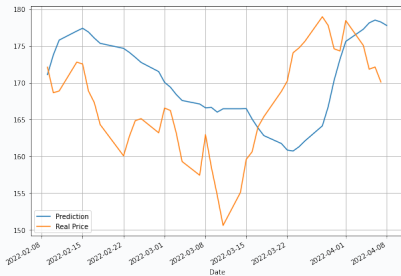
Predicting stock prices with RNNs

We will practice our implementation of RNNs with a prediction problem already familiar to us: **future evolution of a stock!**

- ▶ In this case, we will predict **prices** (rather than returns). Indeed, it makes sense that prices follow a sort of 'sequence' so that we can predict the next day's price by learning the sequence with RNNs.
- ▶ We build an RNN model to predict $d + 1$ price given a sequence of up to 20 past obs. on input prices.

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 50)	2600
dropout (Dropout)	(None, 20, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 50)	5050
dropout_1 (Dropout)	(None, 20, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 20, 50)	5050
dropout_2 (Dropout)	(None, 20, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 10)	510
dense_1 (Dense)	(None, 1)	11

Total params: 18,271
Trainable params: 18,271
Non-trainable params: 0



Summary of Lesson 2

In Lesson 2, we:

- ▶ Implemented our first RNN
- ▶ Tested the performance of RNNs in predicting stock prices

⇒ **References for this lesson:**

Chapter 9, Sections 9.5 and 9.6 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: Go to the accompanying Python Notebook to implement your first RNN!

In the next lesson, we will improve different aspects of the network and also explore in depth the details of forward and backpropagation in RNNs.

Module 4: Lesson 3

Backpropagation in RNNs



Outline

- ▶ Backprop in RNNs: Clipping gradients
- ▶ Dealing with backprop through time
- ▶ Backprop for RNNs in detail

Backprop in RNNs: Clipping gradients

Backpropagation in recurrent neural networks (RNNs) poses a bunch of challenges. We will use this lesson to identify them and offer some simple (and not so simple) ways to mitigate them.

- ▶ The length of the sequence in RNNs introduces a new dimension for our deep neural networks.
- ▶ This means that in each iteration in backpropagation we have to calculate different gradients through time.
- ▶ The consequence is a potentially huge chain of matrix-products that may result in numerical instability.
- ▶ The latter will cause something that, unfortunately, we are already familiar with: **vanishing and exploding gradients**.

A usual 'hack' used in practice to avoid the vanishing/exploding gradients problem is **gradient clipping**:

- Is essentially a way to force gradients to take smaller values.
- In each update of the gradient: $x \leftarrow x - \eta g$, g will be 'clipped' by:

$$g \leftarrow \min \left(1, \frac{\theta}{\|g\|} \right) g$$

Dealing with backprop through time

Applying backpropagation in RNNs is called **backpropagation through time**. Suppose we are in a simple RNN where hidden and output layers at time t are defined as:

$$h_t = f(x_t, h_{t-1}, w_h) \quad \text{and} \quad o_t = g(h_t, w_o)$$

The loss function across T time steps is:

$$L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t)$$

Backprop requires us to compute several gradients, for which we can apply the chain rule:

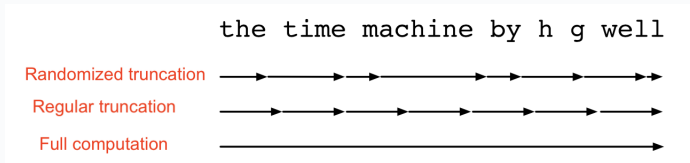
$$\frac{\partial L}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \frac{\partial h_t}{\partial w_h}$$
$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}$$

While the chain rule is very useful for the last computation, it can get very long as t increases!

Dealing with backprop through time

There are a bunch of strategies that we can use to deal with the computation of gradients and chain rules when t is very large:

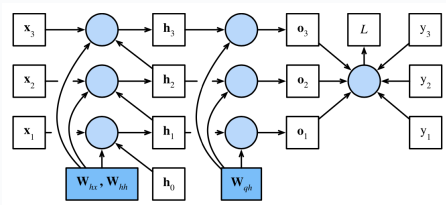
- ▶ Full computation
→ Compute 'the full sum', being aware that things can 'blow up' if t is large
- ▶ Truncating time steps
→ Truncate the sum after τ steps
- ▶ Randomized truncation
→ Use a random variable, which is correct in expectation but truncates the sequence



Backprop for RNNs in detail

The detailed calculations embedded in backprop for RNNs is described in detail in section 9.7.2 in chapter 9 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

- ▶ We direct you to that section (part of the required readings) for the full explanation.
- ▶ Here, we simply review a few of the steps part of the process:



Even in this very simply network we will have to, using the chain rule, calculate (in backprop order):

$$\frac{\partial L}{\partial o_t} \rightarrow \frac{\partial L}{\partial W_{qh}} \rightarrow \frac{\partial L}{\partial h_t} \rightarrow \frac{\partial L}{\partial W_{hx}} \rightarrow \frac{\partial L}{\partial W_{hh}} \rightarrow \dots$$

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Identified the sources of vanishing/exploding gradient problems in RNNs
- ▶ Presented several options to deal with long sequences in RNNs
- ▶ Gone through backprop through time in detail

⇒ **References for this lesson:**

Chapter 9, Sections 9.5.3 and 9.7 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will improve different aspects of the RNN network and revisit the example from Lesson 2, extending it to the multivariate case.

Module 4: Lesson 4

Multivariate RNNs

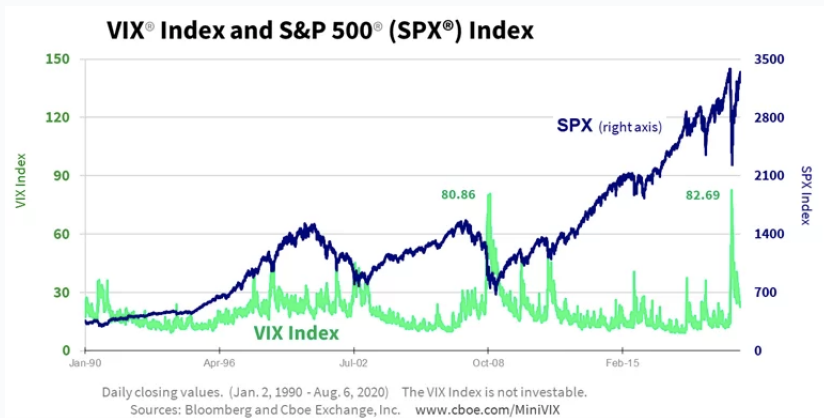


Outline

- ▶ The S&P500 and the VIX
- ▶ Multivariate RNNs

The S&P500 and the VIX

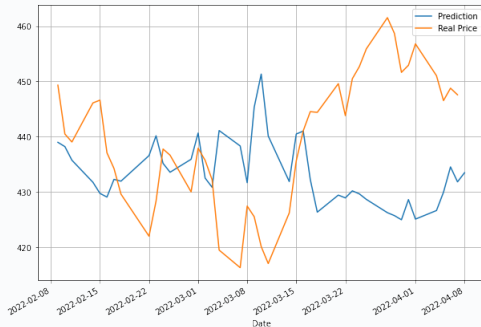
For our next prediction problem, we will use a multivariate RNN. What does this mean? Simply that it contains more than one input feature. This makes a lot of sense, especially if we can get our hands on 'leading' data:



The S&P500 and the VIX

In our multivariate RNN, we will aim to predict the next day's price of the SPY (ETF over the S&P500 index) using past prices of SPY and prices (or levels) of the VIX.

Sadly, our results are not so good...



Note that the training of this network requires some random initialization so the graph may look different!

Summary of Lesson 4

In Lesson 4, we:

- ▶ Explored the relationship between a market index such as S&P500 and its volatility index (VIX)
- ▶ Used both inputs in an RNN
- ▶ Worked with the multivariate case of RNNs

TO DO NEXT: Go to the accompanying Python notebook to implement your first multivariate RNN!

This is all for Module 4. In the next module, we will deal with two 'modern' types of RNNs: gated recurrent units (GRUs) and long short-term memory (LSTM). See you there!